Sistema distribuido para la resolución de problemas SAT

Memoria

Universitat de Lleida
Escola Politécnica Superior
Grau en Enginyeria Informática

Autor:
Jose Miguel Avellana López
Codirector:
Josep Luis Lérida Monsó
Codirector:
Josep Argelich Romá

1 Conceptos previos

1.1 Sistemas distribuidos

Un sistema distribuido es un sistema de software cuyos componentes están separados físicamente y conectados entre sí por una red de computadoras, se comunican y coordinan entre ellos pasando mensajes. Dichos componentes interactúan entre ellos para lograr una meta común. Los sistemas distribuidos permiten la disponibilidad y la facilidad de acceso a la información, trasferencia de datos a grandes velocidades, control al acceso a transmisión de información y administran e integran información.
Los inicios de esta tecnología comenzó en los años 70, usa de las principales características de los sistemas distribuidos eran los mainframes (super computadoras de alta capacidad y eficiencia), pero en esta época se usaban redes WANDS, de transmisión muy lenta, y por tanto el proceso muchas veces se veía limitado por su capacidad de transmisión. Poseían una velocidad de transmisión de 2400 a 9600 bits por segundo. En los años 80 se añadieron los PC de escritorio y se comienzan a usar redes LANS, estas permitieron velocidades de transmisión de 4 a 16 millones de bits por segundo (casi 2 megas.) y de 56 kilobytes a 1.54 megabytes en redes WANS. Durante los años 90 los sistemas de proceso de información se vuelven mucho menos cerrados, lo que permite hacer sistemas distribuidos con diferentes máquinas de diferentes fabricantes conectadas por red desde diferentes áreas geográficas. Además se implementa un gran avance llamado Procesamiento Cooperativo distribuido, este permite el intercambio de la información interactivamente entre los procesadores de diferentes computadores, por ejemplo, la computación cliente-servidor. Las redes de transferencia de datos en esta época son iguales o mayores a 100 megas para redes LAN, MAN y WAN. Actualmente los sistemas distribuidos abarcan desde redes de ordenadores personales donde compartir información, como BitTorrent, o sistemas monetarios, como Bitcoin, hasta los grandes centros de datos que sobre los que funcionan los servicios de Internet más usados, ya que en estos centros se conectan gran cantidad de ordenadores entre sí.

También se han dado grandes avances en la capacidad del software para gestionar gran cantidad de recursos sobre sistemas distribuidos, permitiendo una mayor tolerancia ante fallas bizantinas (resistencia ante la caída o mal comportamiento de parte de la red) y un balanceo más eficiente de la carga de trabajo entre las distintas computadoras de la red.

Parte de esta mejora en el software de gestion de recursos en sistemas distribuidos se debe al concepto de virtualización. Este consiste en imitar las características del hardware y crear un sistema informático virtual, permitiendo ejecutar más de un sistema operativo o aplicación por servidor. La virtualización ofrece una serie de beneficios sobre los sistemas clásicos como una una mayor seguridad ante accesos no confiables a través de firewalls o el aislamiento de los hilos de ejecución y la memoria del sistema ante sistemas maliciosos.
Sobre este concepto nació otro, la contenerización de software, este es un método de virtualización de nivel de sistema operativo (nivel OS) para implementar y ejecutar aplicaciones distribuidas sin lanzar una máquina virtual completa (VM) para cada aplicación. En lugar de virtualizar sistemas independientes, varios sistemas aislados se ejecutan en un único host de control y acceden a un único kernel. Los contenedores de aplicaciones contienen los componentes, como archivos, variables de entorno y bibliotecas, necesarios para ejecutar el software deseado. Debido a que los recursos se comparten de esta manera, se pueden crear contenedores de aplicaciones que ponen menos presión a los recursos globales disponibles. Por ejemplo, si se desea una variación de la imagen estándar (una imagen es un archivo que representa el contenedor), se puede crear un contenedor que contenga solo la nueva biblioteca. En la actualidad son muchos los centros de datos y supercomputadores,

ambos sistemas distribuidos compuestos por diversas máquinas, que se basan en esta tecnología, la contenerización, a la hora de optimizar sus recursos, balanceando la carga de los distintos contenedores entre sus nodos. Los sistemas más notorios sobre este campo son Docker, el cual permite automatizar el despliegue de contenedores, y Kubernetes, que permite, además, escalar y balancear la carga de contenedores en un sistema distribuido.

En un servidor tradicional, todos los componentes de una aplicación son desarrollados y ejecutados sobre una misma base (sistema operativo, plataforma, ...), lo que provoca una menor adaptabilidad y una mayor rigidez del sistema, este tipo de arquitectura se denomina monolítica. En contraste, la contenerización de aplicaciones en sistemas distribuidos ha permitido paso a nuevos tipos de arquitecturas sobre la manera de usar servicios de software, ya sean microservicios, FaaS o Serverless. Los microservicios son tanto un estilo de arquitectura como un modo de programar software. Con los microservicios, las aplicaciones se dividen en elementos más pequeños e independientes entre sí, permitiendo a cada uno de sus componentes ser desarrollados de manera aislada y ejecutados en contenedores sobre sistemas distribuidos, adoptando asi las ventajas de estas tecnologías anteriormente argumentadas.
La función como servicio, o FaaS, es un modelo de ejecución informática que se basa en eventos y se ejecuta en contenedores sin estado. Esas funciones son las que gestionan la lógica y el estado de los servidores a través del uso de servicios. Gracias a la FaaS, los desarrolladores pueden diseñar, ejecutar y gestionar esos paquetes de aplicaciones como funciones, sin tener que mantener su propia infraestructura. La FaaS es una forma de implementar la informática sin servidor (Serverless), donde los desarrolladores escriben la lógica empresarial que luego se ejecuta en los contenedores de Linux totalmente gestionados por una plataforma. Las plataformas más destacadas son AWS, Google Cloud o Azure.
Ambos paradigmas son, habitualmente, complementarios, permitiendo una mayor flexbilidad en el desarrollo y despliegue de aplicaciones.

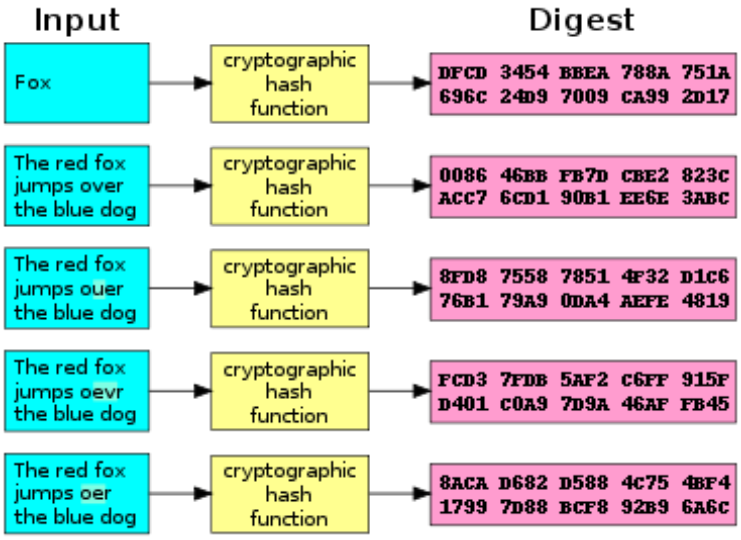
1.2 Funciones hash

Una herramienta criptográfica destacable en muchos sistemas distribuidos, sobre todo si no hay confianza, son las funciones hash. Las funciones de resumen o funciones hash son algoritmos que consiguen crear a partir de una entrada (ya sea un texto, una contraseña o un archivo, por ejemplo) una salida alfanumérica de longitud normalmente fija que representa un resumen de toda la información que se le ha dado (es decir, a partir de los datos de la entrada crea una cadena que solo puede volverse a crear con esos mismos datos). Estas se encargan de representar de forma compacta un archivo o conjunto de datos que normalmente es de mayor tamaño que el hash independientemente del propósito de su uso.

Este sistema de criptografía usa algoritmos que aseguran que con la respuesta (o hash) nunca se podrá saber cuales han sido los datos insertados, lo que indica que es una función unidireccional. Sabiendo que se puede generar cualquier resumen a partir de cualquier dato nos podemos preguntar si se podrían repetir estos resúmenes (hash), la respuesta es que teóricamente si, podría haber colisiones, ya que no es fácil tener una función hash perfecta (que consiga que no se repita la respuesta), pero esto no supone un problema, ya que si se consiguieran ante dos hashes iguales los contenidos serían totalmente distintos (con un buen algoritmo).

Por tanto, las características que hacen que una función hash sea util en este trabajo es que son deterministas y unidireccionales. De manera que con una misma entrada siempre se obtiene la

misma salida, con la salida no se puede obtener la entrada (facilmente) y una pequeña modificación de la entrada supone una salida totalmente distinta. Permitiendo de esta manera la identificación e integridad de la información en sistemas distribuidos sin confianza entre sus pares.



1.3 Problema SAT

Los problemas NP-completo son los más complicados de la clase NP, en el sentido que si Q' es un problema de decisión en NP y Q es un problema NP-completo, entonces todas las instancias de Q' son polinomialmente reducibles a una instancia de Q. El problema de satisfacibilidad (SAT) fue el primer problema identificado como perteneciente a la clase de complejidad NP-completo por Stephen Cook en el año 1971. El problema consiste en saber si, dada una expresión booleana con variables y sin cuantificadores, hay alguna asignación de valores para sus variables que hace que la expresión sea verdadera. Por ejemplo, una instancia de SAT sería el saber si existen valores para x_1, x_2, x_3, $x_4 x_1$, x_2, x_3, x_4 tales que la expresión:

$$(x_1 \vee \neg x_3) \wedge (\neg x_2 \vee x_3 \vee \neg x_4)(x_1 \vee \neg x_3) \wedge (\neg x_2 \vee x_3 \vee \neg x_4) \tag{1}$$

sea cierta.
Los algoritmos eficientes y escalables para SAT se desarrollaron en la última década y han contribuido a avances dramáticos en nuestra capacidad de resolver automáticamente instancias de problemas que involucran decenas de miles de variables y millones de restricciones (es decir, cláusulas). Sobre esto, un SAT solver es una herramienta que toma como entrada una fórmula CNF y genera una asignación booleana satisfactoria a las variables utilizadas en la fórmula CNF si la fórmula es satisfactible o no.

La clasificación más comun sobre algoritmos que resuelven SAT es si son completos o incompletos. El algoritmo completo busca el espacio de solución de un problema SAT dado basándose en mecanismos de búsqueda de enumeración y retroceso. Si el problema es satisfactorio, entonces se dará la solución de la fórmula lógica, y si no, se dará prueba de que está completo. De hecho, en el segundo caso, es extremadamente importante y práctico dar prueba de integridad, ya que muchos casos insatisfactorios requieren prueba de que no pueden satisfacerse. El algoritmo completo determina el valor de las variables en la lógica proposicional una por una hasta que se asignan todas las variables

y el valor de la fórmula es verdadera, o se intentan todas las asignaciones posibles pero quedan insatisfechas. La ventaja del algoritmo completo es que puede garantizar la solución del problema SAT correspondiente o probar que la fórmula no es satisfactoria. Desafortunadamente, la eficiencia del algoritmo completo es extremadamente baja. Aunque la complejidad de tiempo promedio es un nivel polinomial, la complejidad de tiempo del peor de los casos es exponencial. El algoritmo completo es un proceso de búsqueda en profundidad de todo el espacio de la solución, el espacio de búsqueda es muy grande, lo que puede hacer que la computadora no pueda devolver los resultados en un tiempo aceptable.

Por otro lado, el algoritmo de búsqueda local es el más utilizado en el algoritmo incompleto. Este método utiliza cierta estrategia para buscar aleatoriamente en el espacio de asignación de la variable, y se acerca gradualmente a la solución final bajo la guía de la función objetivo. Hay dos tipos de funciones objetivo: una es minimizar el número de cláusulas con el valor en falso, y la otra es satisfacer el número máximo de cláusulas de restricción. La ventaja del algoritmo incompleto es una mayor eficiencia a la hora de encontrar interpretaciones válidas siempre y cuando existan, de no ser asi el algoritmo nunca dará con una solución.

Otra forma de resolver problemas SAT es transformar el problema de satisfacibilidad en otro problema en otro campo, que se puede resolver rápidamente, como el algoritmo genético, el algoritmo de recocido simulado o la propagación de encuestas.

2 Introdución

2.1 Contexto

En 1970, el matemático británico John Horton Conway diseño un automata celular al que llamo El Juego de la Vida. Un autómata celular es un modelo matemático y computacional para un sistema dinámico que evoluciona en pasos discretos.
El juego de la vida viene a decir que, a partir de un conjunto de reglas simples, se pueden generar estructuras complejas. Un sistema compuesto de un conjunto de agentes (o células), que ocupan un lugar en él, que poseen una serie de reglas que definen como interaccionan con el resto de agentes. Y es que, en cierto modo, asi es como funciona la evolución de un ecosistema de seres vivos, mejorando y optimizando sus procesos sin que exista una intención consciente por su parte.

Bajo este mismo principio se puede entender la capacidad de los sistemas económicos basados en la competencia para mejorar sus procesos. Cuando una empresa se dedica a investigar nuevas tecnologías para abaratar costes, no lo hace por el bien común, lo hace para vencer a sus rivales, pero, de manera inconsciente, sirve al bien común. Será el mercado el que dictará si esta nueva forma de realizar dicha tarea es más optima o no.
Por lo tanto, es la competencia de ideas, de procesos y formas de actuación la que nos permite mejorar la forma en que vivimos. Teniendo en mente, por ejemplo, procesos industriales o logísticos, si a alguien se le ocurre una forma más eficiente de hacer algo creará una empresa que realice esta labor en la manera que propone, si realmente es mejor que el método tradicional, terminará abarcando un cada vez mayor porcentaje de su mercado e imponiendose al método anterior.
Pero esta es una forma muy lenta de evolucionar debido a que el tiempo para observar resultados es el de un escenario real. Sin embargo, se pueden observar nuevas propuestas de actuación en entornos simulados, en mucho menor tiempo y barajando más opciones, y de esta forma aplicar la

mejor forma de actuación practicamente al instante.

La revolución informática de las ultimas décadas nos da la posibilidad de simular y automatizar procesos. Se ha construido una red de ordenadores con estas capacidades, y su potencial es enorme. Tecnologías de vanguardia como la inteligencia artificial nos prometen una significativa mejora en nuestra calidad de vida, principalmente por su potencial para automatizar tareas como nunca antes se había imaginado.

En un automata celular, a menor número y/o complejidad de las reglas sobre la interacción de sus agentes mayor será el espectro de complejidad que pueda alcanzar el autómata. Es decir, contra más reglas tiene más se limita.

2.2 Motivación

Por ello se considera interesante el analisis y diseño de un sistema compuesto por servicios, entendidos como formas de actuación determinadas para resolver un problema concreto, donde prevalezca siempre el más competitivo y eficiente, ejecutados sobre una red de nodos sin confianza gracias a la tecnología previamente mencionada. El sistema contará con un conjunto de servicios que resuelven un problema determiando además de otro que se dedica a compararlos o clasificarlos. Como problema a resolver se escoje el de satisfactibilidad (SAT) por ser un problema interesante en el campo de la IA.
De forma que, ante la necesidad de resolver SAT, no sea necesario escojer un algoritmo concreto (forma de actuación determinada) si no que el sistema escoja la mejor de forma autónoma incluso con el paso del tiempo, pudiendo el usuario añadir posteriormente nuevos algoritmos (SAT-solvers) que compitan con el resto.

2.3 Objetivos

Por tanto el objetivo general del proyecto es diseñar un servicio que clasifique y seleccione el SAT solver más eficiente para un CNF concreto. Este trabajo pretende, principalmente, dos cosas:

- Definir, de manera simple, las reglas básicas de cada uno de los componentes de un sistema distribuido de servicios, y como se definen e interaccionan entre sí.
- Dar uso del sistema para realizar una prueba concepto sobre un problema concreto, la resolución de problemas SAT.

El sistema debe de, en última instancia, ser capaz de resolver un problema SAT escogiendo, en función del CNF de entrada, el más eficiente de una serie de algoritmos que lo resuelvan. Sistema en el cual, tanto el clasificador como cada uno de los algoritmos (SAT solvers) son servicios independientes i aislados interaccionando en un entorno sin confianza, los unos con los otros y con los nodos, los cuales, además, se encargan de balancear su carga con el resto de nodos sin que exista un nodo central que supervise el balanceo.

Con esto debería de ser posible acoplar nuevas funciones o características, tanto a los nodos como a los servicios, sin que el resto de componentes se vieran afectados. Por ejemplo, añadir un nuevo algoritmo para resolver SAT, añadir un sistema de clasificación distinto o modificar las políticas de balanceo de carga de uno de los nodos.

3 Estado del Arte

Como se ha expuesto con anteriordad (conceptos previos), actualmente los problemas SAT se resuelven, principamente: con algoritmos completos, incompletos o transformando el problema en otro. El rendimiento o eficiencia de un algoritmo puede variar segun el tipo de CNF. Por tanto se pretende clasificar cualquier tipo de los algoritmos que resuelven SAT anteriormente mencionados a partir del tiempo que tardan en resolver un CNF concreto, es decir, en función del rendimiento que poseen ante un tipo u otro de CNF (de mayor o menor número de clausulas o literales). Por ejemplo, que en un algoritmo incompleto prevalezca un determinado grado de aleatoriedad frente a otro o una función objetivo ante otra puede brindarle una mayor probabilidad de éxito ante CNF's con un determinado número de clausulas y literales, una determinada proporción entre el número clausulas y literales o el número de literales por cláusula. Por tanto, saber de antemano que algoritmo obtuvo un mejor rendimiento con un CNF similar podría brindar una ventaja al seleccionar el más oportuno.

Para realizar la tarea de clasificación se opta por dar uso del método de regresión lineal. La regresión lineal es un algoritmo de aprendizaje automático basado en el aprendizaje supervisado. Realiza una tarea de regresión. La regresión modela un valor de predicción objetivo basado en variables independientes. Se utiliza principalmente para averiguar la relación entre las variables y la previsión y puede ayudar a predecir el comportamiento de sistemas complejos o a analizar datos. Los diferentes modelos de regresión difieren en función del tipo de relación entre las variables dependientes e independientes, que están considerando y el número de variables independientes que se utilizan.

Las técnicas de regresión lineal permiten crear un modelo lineal. Este modelo describe la relación entre una variable dependiente y (también conocida como la respuesta) como una función de una o más variables independientes χ_i (denominadas predictores). La ecuación general correspondiente a un modelo de regresión lineal es:

$$Y = \beta_0 + \sum \beta_i \chi_i + \epsilon_i \tag{2}$$

donde β representa las estimaciones de parámetros lineales que se deben calcular y ϵ representa los términos de error.
En el caso de la clasificación de algoritmos SAT (o SAT solvers) la variable dependiente del modelo sería el grado de eficiencia del algoritmo y las variables independientes el número de clausulas y literales del CNF.

El servicio que se encarga de realizar la regresión lineal da uso de la librería de Sklearn. Scikit-learn es una biblioteca para aprendizaje automático de software libre para el lenguaje de programación Python, incluye varios algoritmos de clasificación, regresión y análisis de grupos.

En el contexto de la los sistemas distribuidos y la computación en la nube, la virtualización es una técnica que crea un ecosistema virtual de dispositivos de almacenamiento y sistemas operativos del servidor, facilitando la escalabilidad y reduciendo la necesidad de un sistema físico. Desde la externalización del hardware utilizado y la reducción de la implementación de fuentes específicas, hasta la eliminación de costos adicionales de energía, la virtualización es una técnica muy utilizada en este tipo de entornos (sistemas distribuidos y computación en la nube).

Para realizar la tarea de construcción y gestion de servicios, virtualización o contenerización de cada uno, se da uso de la plataforma Docker, principalmente por ser la más extendida y con mayor comunidad en su nicho. Docker es un proyecto de código abierto que automatiza el despliegue de aplicaciones dentro de contenedores de software, este permite construir y ejecutar cada uno de los

servicios que componen todo el sistema. Otra plataforma muy usada en el contexto de la contenerización de software es Kubernetes, esta puede integrase con Docker para escalar y balancear contenedores en red.

4 Análisis de requisitos

Con el fin de garantizar un desarrollo del trabajo eficiente se realiza antes un analisis de los requisitos que debe de cumplir el sistema. Concretando los objetivos expuestos anteriormente, estos son :

- Diseño de la especificación de un servicio.
- Desarrollo de un nodo con capacidad para:
  - Compilar, construir y ejecutar servicios de manera autónoma.
  - Compartir servicios con el resto de los nodos de la red y solicitarlos cuando no disponga de ellos.
  - Balancear la carga de las instancias de servicios.
- Desarrollo de un servicio de clasificación y selección de servicios que resuelven SAT. Este debe ser capaz de:
  - Cargar especificaciones de SAT-solvers, pedirle al nodo que le devuelva instancias de los mismos y mantenerlos (comprobar que siguen activos).
  - Clasificar los SAT-solvers que se le han especificado de manera que asegure una buena selección de estos.
  - Seleccionar un SAT-solver en función de un CNF dado para que lo resuelva y retornar la solución.
  - tolerar y controlar errores y excepciones por parte de otros nodos, sin que estos afecten la ejecución de las instancias que él mismo ejecuta.

Sobre estos objetivos se pretende analizar más a fondo los requisitos a cumplir en cada uno.

4.0.0.1 Requisitos de la especificación de un servicio Debe de ofrecer la capacidad de definir e identificar el servicio mediante un objeto estructuralmente definido. Saber que partes lo componen, que ofrece y como usarlo. De manera que pueda ser identificado mediante una función hash determinista, para ello se debe de acordar como obtener el identificador de la especificación.

4.0.0.2 Requisitos de un nodo Un nodo debe de ofrecer:

- compilar la especificación de un servicio a partir de un conjunto de datos o archivos de entrada.
- una interfaz de aplicación que permita comunicarse con él, tanto por parte de los servicios que instancia como por parte de otros nodos de la red.
- instanciar un servicio a partir de su especificación de manera autónoma, es decir, cargar la especificación, comprenderla, construir el contenedor si no está construido y ejecutarlo según se indique en la especificación, de manera que el solicitante pueda dar uso de la instancia directamente.
- compartir sus servicios y las especificaciones de estos.

- determinar que coste puede suponer ejecutar la instancia de un servicio.
- comprobar un posible nuevo nodo en la red a partir de la dirección de este y validarlo.
- atender la solicitud de comprobación de otros nodos que pretenden verificarle.

Además de ello, debe ser capaz de:

- balancear la ejecución de las de instancias de servicios que se solicitan entre sí mismo y el resto de nodos de la red. Buscando la mayor eficiencia de sus recursos.
- permitir que la política de balanceo sea independiente con el resto de la red, es decir, sin que exista una comun y sin obedecer un nodo central.
- controlar las instancias que está ejecutando y las instancias que ordenó ejecutar, asi como que instancias ha solicitado cada una de ellas.
- controlar un uso adecuado de la memoria, es decir, no seguir ejecutando instancias que ya no se usan.
- encontrar nuevos nodos en la red sin configuración previa, es decir, sin que deba de introducirse su dirección para que posteriormente lo busque.
- cuando reciva la especificación de un servicio junto con su identificador, validar la integridad de esta.
- tolerar y controlar errores y excepciones por parte del nodo y de los servicios SAT-solver.

4.0.0.3 Requisitos del servicio de clasificación y selección El servicio principal debe de ofrecer:

- una interfaz de aplicación que permita comunicarse con él.
- añadir especificaciones de servicios que resuelvan SAT.
- determinar si se debe de iniciar o detener el proceso de entrenamiento de los SAT-solvers.
- resolver el problema SAT a partir de un CNF de entrada.
- añadir datos para sumar al entrenamiento.
- la obtención de los datos de entrenamiento y/o el modelo entrenado a partir de estos.
- la obtención de los 'logs' del sistema en tiempo real.

Además de ello, debe ser capaz de:

- permitir solicitudes de manera concurrente.
- resolver SAT usando el SAT-solver más eficiente en función del CNF de entrada y los datos recogidos durante el entrenamiento.
- resolver SAT en un tiempo similar al que necesitan los SAT-solvers usados de manera directa, sin que estos últimos tengan tiempos de respuesta de varias unidades por debajo, es decir, sin que el tiempo en resolver SAT sea la más de la mitad en cualquiera de los SAT-solvers entrenados al tiempo en que tarda el servicio en seleccionar uno de ellos y resolver el problema.

- almacenar las especificaciones de los servicios recibidos.
- comunicarse con un nodo (sea el que lo ejecuta o no) y poder solicitarle que ejecute y detenga las instancias de los servicios SAT-sorter.
- controlar que instancias de SAT-solvers se ejecutan correctamente y cuales no, además de solicitar detenerlas y ejecutarlas de nuevo cuando sea necesario.
- ejecutar más de una instancia de un mismo SAT-solver si asi se requiere y, con esto, ser capaz de balancear la carga de solicitudes de ese mismo SAT-solver entre sus múltiples instancias.
- asegurar una cierta eficiencia a la hora de solicitar nuevas instancias, de manera que no provoque la sobrecarga del nodo o los nodos de manera innecesaria.
- generar CNF's aleatorios con los que provar los SAT-solvers durante el entrenamiento de datos.
- recolectar datos del entrenamiento de manera eficiente, permitiendo realizar el entrenamiento durante largos periodos de tiempo sin generar una perdida de rendimiento del mismo.
- al recibir nuevos datos de entrenamiento por parte del cliente, validarlos y adjuntarlos al set de datos principal en función de su integridad o calidad, de manera que no requiera confianza hacia el cliente sobre los datos proporcionados.

5 Definición y desarrollo

5.1 Planificación temporal del trabajo

El trabajo se ha dividido en las siguientes partes, tal y cómo se puede ver en el diagrama de Gantt de la figura XXX.

1. Estudio previo y definición del sistema
   - Definir las responsabilidades que tienen tanto los nodos como los servicios.
   - Concretar que debe de especificar la definición de un servicio, como se obtiene el identificador del mismo y el tipo de serialización que se debe de usar.
2. Reunion inicial
   Se evalua la idea del trabajo en terminos generales y se estudian los requisitos y tiempos estimados del proyecto.
3. Reunion de seguimiento.
   Se analizan las tareas ya realizadas del trabajo, se reajusta la planificación temporal y se debate como llevar a cabo el balanceo de instancias y solicitudes.
4. Reunión final.
   Se estudian y analizan las conclusiones y resultados del trabajo.
5. Fase de desarrollo del nodo
   Desarrollar las funciones básicas de un nodo:
   - Compilar a la especificación de un servicio.

- Construir la especificación de un servicio.
  Dada la especificación de un servicio, debe ser capaz de construir un contenedor del mismo, abrir los puertos necesarios y añadirle las variables de entorno dadas, si es el caso.
- Servicio local con la finalidad de atender las peticiones de los servicios.
  Las peticiones básicas son: la inicialización de un servicio, y su detención. Al especificarle un servicio, el servicio local del nodo le devolverá al cliente la dirección de la instancia del servicio demandado y un token con el que efectuar acciones u obtener información sobre él (como detenerlo, por ejemplo).
- Balanceo de carga de instancias.
  Al ser el nodo el que tiene la responsabilidad de construir y ejecutar los servicios demandados, este puede derivar a otros nodos de la red la tarea de construir y ejecutar un servicio.
  Se debe de desarrollar la lógica necesaria para que lo haga.
- Distribución de contenedores.

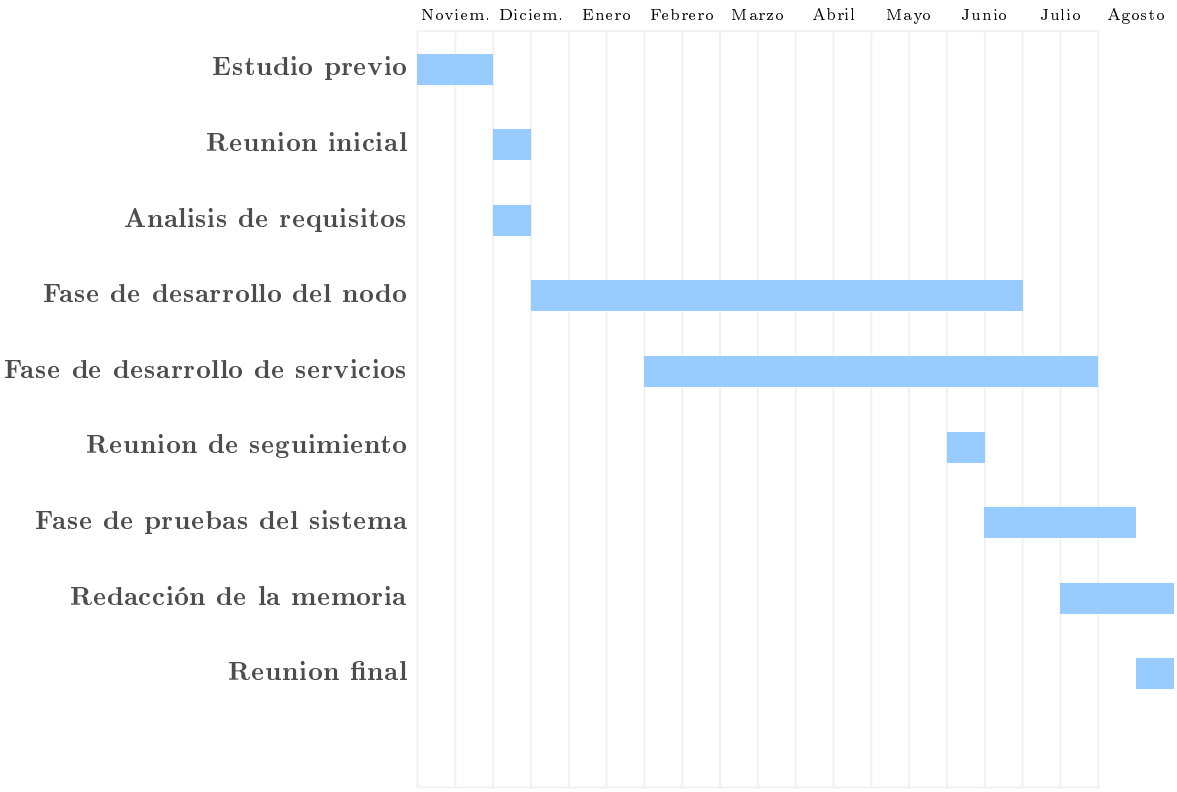
- Busqueda de otro nodos en la red.

6. Fase de desarrollo de los servicios

   - Desarrollar servicios SAT-solver y sus especificaciones.
   - Desarrollar un servicio de clasificación de SAT-solvers.
     Por un lado se ha de desarrollar un sistema que genere CNFs aleatorios, pruebe cada uno de los solvers y recoja datos de ello. A partir de los datos obtenidos previamente se deben clasificar los solvers mediante algun tipo de regresión.
     Por otra parte, el servicio debe de tener la capacidad de, a partir de un CNF dado, resolverlo dando uso del solver que mejor se adapte a ese CNF en concreto.
   - Balanceo de carga de solicitudes.
     El clasificador deberá tener un balanceador de carga con el que se repartirán las peticiones para resolver CNF's entre varias instancias de un mismo solver. sobre él (como detenerlo, por ejemplo).

7. Fase de testing del sistema
   Se comprueba el comportamiento del sistema en su conjunto, balanceo de carga, conexión entre nodos, respuesta ante la caida de servicios . . . , además de su eficiencia.

Ambas fases de desarrollo se abarcan de manera paralela, su distinción se debe a la abstracción entre nodo y servicio el cual permite un desarrollo de este tipo, es decir, aislar cada parte y desarrollarla sin preocuparse demasiado de la otra.



Durante el desarrollo se han originado una serie de retrasos o inconvenientes frente a la planificación inicialmente estipulada. Estos son, por un lado, el cambio de Json a Grpc y, por otro, le división del sistema de regresión lineal del modelo del servicio principal.

La versión inicial del sistema contó con el uso de REST y Json como estilo y sistema de serialización tanto en la comunicación servicio-nodo como en la especificación de un servicio. El motivo inicial de dar uso de esta tecnología fue su simplicidad y su posición dominante en su nicho (posiblemente el más usado en la comunicación entre sistemas, sobre todo cliente/servidor). Una vez observada la capacidad del servicio de clasificación y el nodo para interactuar entre sí usando esta arquitectura/lenguaje, aunque esta tecnología si que cumplía con los requisitos establecidos, se decidió estudiar que otras tecnologías podrían ser más convenientes en el contexto de un sistema de microservcios. Finalmente se decidió dar uso del sistema de comunicación Grpc y se adaptó todo el sistema desarrollado hasta ese momento al mismo. Más adelante, en la sección '6. Problemas encontrados y decisiones técnicas tomadas' se profundiza sobre este nuevo enfoque y las ventajas e inconvenientes que conllevó.

La separación del sistema de regresión del servicio principal fue motivada y ejecutada de la misma forma que la decisión anterior, una vez que la regresión cumplía con los requisitos marcados anteriormente dentro del servicio principal se estudiaron diversas arquitecturas (más complejas que tener toda la lógica en el mismo servicio) con las que poder sumar eficiencia al sistema. Esto es principalmente porque si uno de los motivos de ejecutar microservicios en un sistema distribuido es repartir el coste computacional entre todos los nodos, tener un servicio principal que recoge datos, entrena modelos, balancea solicitudes y clasifica es ir, en cierto sentido, en dirección opuesta. Por tanto separar la regresión, que es posiblemente la tarea más costosa en términos computacionales

de todo el sistema, debería ayudar a balancear la carga entre los nodos y por tanto ganar eficiencia. Además de esto, desarrollar e integrar un nuevo servicio en el sistema sin la necesidad de grandes cambios estructurales demuestra la capacidad de escalabilidad de la arquitectura de un sistema basado en microservicios.

Por otra parte, hay dos requisitos que no han sido cubiertos durante el desarrollo e implementación del proyecto. Uno de ellos es la capacidad de un nodo de encontrar pares en la red sin configuración previa, en lugar de esto, al añadir un nuevo nodo, es necesario informar a cada uno del resto de nodos de la red con la dirección de este, característica también especificada en los requisitos de un nodo. Por lo que quedaría como trabajo futuro implementar una política de configuración cero.
Otro requisito no abordado es el análisis, por parte del servicio de clasificación, de los datos proporcionados por el cliente. La forma en la que se ha implementado requiere tener confianza en el cliente sobre los datos proporcionados, el requirimiento establecia la necesidad de no confiar, implementando algún tipo de prueba sobre los datos que les otorgara mayor o menor peso sobre el set de datos principal. Por lo tanto, de igual manera, quedaría como trabajo futuro implementar esta función.

5.2 Definición del sistema

Los conceptos principales que se han definido son:

- Un servicio es un contenedor de software que realiza una tarea especifica, puede entenderse como una "caja negra", existe una especificación del mismo en formato binario, donde se especifica todo lo necesario para construirlo y comunicarse con él que hace innecesario conocer su funcionamiento interno. Puede tener o no estado, pero esto es irrelevante en la especificación.

- Un nodo es un computador que forma parte de la red, debe de tener la capacidad de encontrar otros nodos y comunicarse con ellos, compartir y construir servicios y orquestar los servicios entre él y otros nodos.

- El término instancia se refiere a la instancia de un determinado servicio, es decir, un contendor que se ejecuta siguiendo los critérios de la especificación de dicho servicio.

- Tanto los servicios como los nodos ejecutarán servidores con los que atender solicitudes, concretamente en el trabajo son servidores Grpc.

- La especificación de un servicio se refiere a un buffer de bytes que define el servicio, esto es, como esta formado y como comunicarse con él, y no se refiere a como construirlo. La especificación de un servico se serializa en un binario Protobuf.

- La imágen de un servicio se refiere a la imágen del contenedor de un servicio, asi como un contendor se refiere a la instancia de la imagen de un servicio.

- El concepto implementación del nodo pretende remarcar aquellas decisiones sobre la lógica del nodo que son independientes del funcionamiento general de la red y, por tanto, otros nodos pueden no adoptar.

- La configuración de un servicio se refiere a unas variables de entorno determinadas de este. Sirve para identificar una 'versión' concreta de un mismo servicio.

- Una instancia zombie es aquella que o bien cuyo servicio padre, quien la solicitó, ha perdido su token y por tanto no puede matarla, o bien el servicio que la solicitó ya ha muerto.

Definir como debe comportarse un sistema requiere abordar cuales son los deberes y derechos de sus componentes.
Un nodo debe de proporcionar una interfaz con la que los servicios puedan comunicarse, junto con la dirección a la que el servicio le realizará sus solicitudes. La forma en la que el nodo carga estos componentes en una nueva instancia es mediante un archivo binario en la raíz del sistema de ficheros del servicio (__config__). Esto es así ya que el servicio podría requerir de estos datos antes de levantar su servidor.
Ante la solicitud de una nueva instancia, un nodo tiene el deber de, o bien construir esta instancia localmente, o bien solicitarle la instancia a un par (otro nodo), y retornar al solicitante tanto una dirección con la que comunicarse con la nueva instancia como un token con el que realizar acciones sobre ella. En caso de crear la dependencia de un servicio en otro nodo, el nodo debe de asegurarse que el solicitante tenga acceso a la dirección que le otorga para comunicarse con su dependencia.
Esta arquitectura permite a un servicio no preocuparse del nodo donde se ejecutará cada dependencia. Del mismo modo que los nodos no se preocuparán de la utilidad que puedan tener las instancias que ejecutan.
Si un nodo posee una función con la que obtener el coste de ejecutar la instancia de un determiando servicio. Sin especificar en que debe de basarse este coste.

Por tanto, para resumir, durante la construcción de la instancia de un servicio, el nodo añade en su raíz un binario que contiene como comunicarse con él. Si el servicio quiere dar uso de una dependencia, le enviará la especificación de ésta al nodo, y éste le devolverá una dirección con la que comunicarse y un token para poder matarla en un futuro. Será el nodo el que busque donde construir esa dependencia, si localmente o en alguno de sus pares. También será el nodo el que, en caso de matar a la primera instancia, acabe con la segunda (ya que es dependencia de la primera).

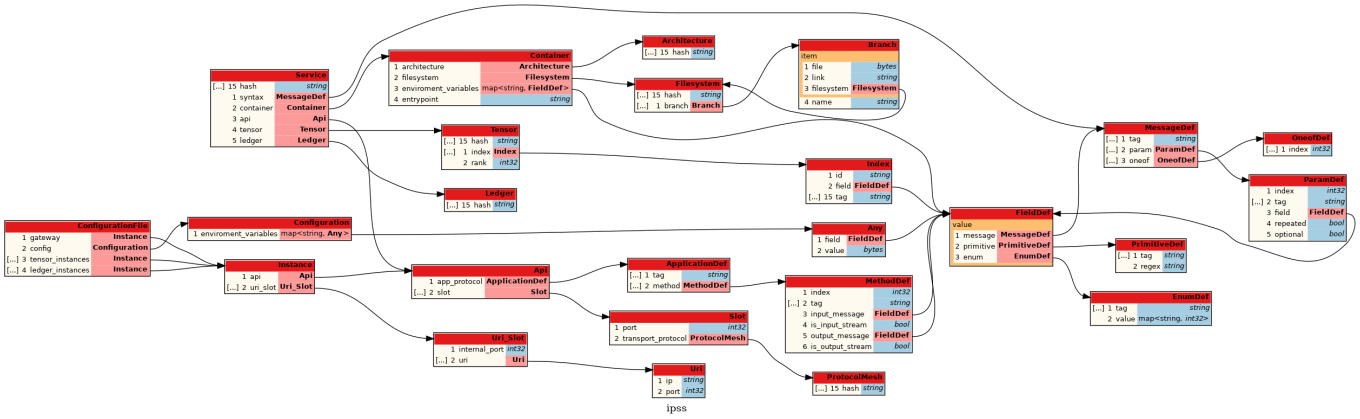
La parte más extensa de la definición del sistema es la especificación de un servicio, este se compone de los siguientes atributos:

- Hashtag: Hashes (del tipo <'sha256' id>:913ref3...) sobre el binario del resto de la especificación.
- Syntax: Sintaxis de la especificación.
- Container: Especifica el contenedor donde se ejecuta el servicio, esto es:
  - la arquitectura, define el hardware y el kernel donde se debe de ejecutar el servicio.
  - el punto de entrada, define la ruta del primer ejecutable del servicio.
  - el sistema de archivos, representado en un árbol donde las ramas pueden ser o bien un archivo binario, o un link a otro elemento del árbol, o otro árbol, y donde cada elemento se identifica con un nombre.
  - las variables de entorno, lista compuesta por llave y definición del campo de cada variable.

  No abarca como se debe de construir (Dockerfile, capas de Docker ...), para no limitar la forma en la que se construyen.
- Api: Define cómo se puede interactuar con el servicio, constituida por:

- interfaz de la aplicación (mensajes y métodos junto con errores y respuestas de cada uno).
- lista de slots. El motivo de requerir slots es el de permitir comunicarse con un servicio en distintos protocolos. Cada slot está compuesto por los siguentes elementos:
  * puerto donde escucha (desde el servicio, de solicitarselo a otro nodo éste podría dar otro).
  * protocolo de comunicación que soporta (HTTP/2, GRPC ...).

- Tensor: Opcional. Definie la topología del modelo que entrena el servicio. Un servicio puede solicitar al nodo que le retorne las instancias de aquellos servicios que tengan un modelo determinado y con ellas realizar solicitudes a servicios que ya tienen el modelo entrenado.



Despues de especifiar los componentes del sistema y a que se dedica cada uno se ha realizado una implementación de cada uno de ellos para materializar la descripción de este.

5.3 Fase de desarrollo del nodo

Esta fase de desarrollo abarca la lógica detrás de un nodo, esto es, la compilación y construcción de servicios y el manejo de solicitudes de los mismos. El entorno donde se ejecuta esta implementación del nodo será Python3 sobre Ubuntu(Linux), dando uso, además, del sistema de base de datos MongoDB, donde se almacenan la información relativa a otros nodos de la red, y el motor de contenerización DockerEngine, con el que ejecutar instancias de servicios. El uso obligatorio de Linux se debe principalmente a que dentro de la lógica implementada en Python se da uso de comandos Bash. Además de eso, se da uso de Systemd para correr tres demonios de manera intermitente e indefinida: docker, mongo y gateway.py. La lógica detras del nodo se puede dividir en tres componentes principales: Compilador, Constructor y Gateway.

Existen, además, dos directorios, registry y hycache, en registry se almacenan las descripciones de servicios provenientes, o bien de su compilación (Compilador) o bien una solicitud de otro servicio interno o externo (Gateway), y el directorio hycache sirve para almacenar datos durante diferentes operaciones, ya sea para almacenar el contenedor comprimido de un servico al ser solicitado a otro nodo (Constructor), guardar la configuracion de una nueva instancia para copiarla dentro de su contenedor (Gateway) o para almacenar y calcular la hash de un sistema de archivos durante la compilación de un servicio (Compilador).

Una vez definidos los componentes principales del nodo se definen más en detalle cada uno:

5.3.0.1 Compilador Ubicado en el archivo compile.py El objetivo de este script de Python es obtener la descripción de un servicio a partir de un directorio llamado .service/, ubicado en la raiz de un repositorio git, que contenga la información necesaria para ello, esto es: un archivo Dockerfile y distintos otra información contenida en un archivo Json.
Por lo tanto, desde la terminal de un nodo, el comando python3 /home/hy/node/compile.py <direccion del repositorio>::<rama del repositorio> generará la descripción del servicio en registry con nombre <sha256 del servicio>.service y la imagen del contendor en el registro de Docker con nombre <sha256 del servicio>.service.
El formato antes de la compilacion (Dockerfile y Json en un directorio .service en la raiz de un repositorio git) tiene como objetivo facilitar la compilación para un desarrollo que de uso de la herramienta Git, la definición del servicio es totalmente independiente de este, por lo que la compilación desde otro formato es facilmente implementable.
El sistema permite la compilación simultánea al generar un identificador aleatorio por cada ejecución, de esta forma el almacenamiento de datos en hycache ocurre dentro de un subdirectorio distinto por cada una. El script ejecuta comandos de docker mediante bash, dando uso del método os.system de Python.

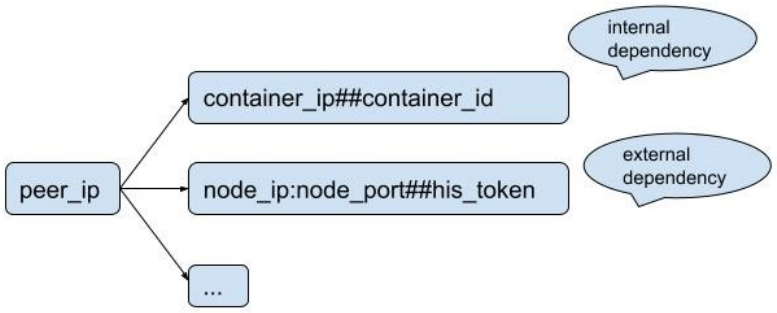
Al iniciarse la compilación se descarga el repositorio aportado en hycache, generar la descripción de un servicio requiere obtener sus atributos y calcular su hash. Gran parte de los atributos fueron almacenados en el archivo Json (Api, Envs, Entrypoint ...), lo único que se debe de hacer es leer el archivo Json y almacenarlos en un objeto Protobuf. Para el sistema de ficheros (que es un subatributo de container) será necesario construir su contenedor mediante el Dockerfile para obtener su sistema de ficheros (esto lo realiza el método parseFilesys), dar uso del método save de docker y comprimirlo en un archivo .tar para posteriormente extraer el sistema de ficheros. Docker divide la imagen de un contendor por capas, aunque cumple su función al contenerizar, es indiferente a la hora de definir el servicio puesto que distintas combinaciones de capas son posibles para un mismo servicio. Es por ello que el archivo .tar donde Docker almacenó el contendor es en realidad un directorio con cada una de las capas comprimidas, para obtener el sistema de ficheros completo se debe de descomprimir cada una y adjuntar todas en el mismo directorio. Finalmente se añade el sistema de ficheros al objeto Protobuf.
Una vez esté el objeto Protobuf completo se puede proceder a calcular las hashes de este para, posteriormente, añadirlas al mismo. Esta implementación del nodo da uso de varios algoritmos hash, aunque es el sha3-256 el que toma como referencia para identificar la descripción y el contenedor en todo el sistema. Tras calcular las hashes se vacía el atributo referente al sistema de ficheros de la especificación del servicio (objeto Protobuf) para no hacerlo tan pesado, no estar completo supone que al compartirlo no será posible verificar sus hashes. Para finalizar la compilación el nodo guarda el objeto Protobuf, referente a la especificación del servicio, en registry y elimina los archivos pertinentes a la operación de hycache (subdirectorio con el identificador generado aleatoriamente al inicio).

5.3.0.2 Constructor Ubicado en el archivo build.py Se compone de un único método (build) el cual recibe la especificación de un servicio (objeto ipss.Service) y construye la imágen docker para permitir generar instancias de este. Tomando la hash sha3-256 de la especificación del servicio (en la llamada al método se debe de asegurar que se encuentra esta hash, de no ser así el método no hace nada) verifica que se encuentre su respectiva imágen en el registro de Docker.

En caso de que no se encuentre la imágen se deberá buscar en otros nodos, por lo que se toman las instancias de otros nodos y dará uso del método GetServiceTar (supone que el resto de nodos lo implementan). El nodo de la red que posea el contenedor se lo devuelve mediante el streaming de lotes del archivo .tar del mismo, los cuales son almacenados en un archivo en hycache. Una vez que se ha descargado todo el archivo Docker se encarga de cargarlo en un contenedor y se le da el nombre correspondiente a su identificador (<hash sha3-256>.service).

5.3.0.3 Gateway Ubicado en el archivo gateway.py Se trata del servidor Grpc encargado de atender las solicitudes tanto de las instancias que ejecuta el nodo como de otros nodos de la red. Su labor principal es ser capaz de generar instancias de servicios y destruirlas mientras gestiona de que instancia o nodo es dependiente cada una, esto es de vital importancia debido a que no puede fiarse del resto de agentes (nodos e instancias) por lo que si un nodo se desconecta o destruye una instancia deberá destruir todas sus dependencias. También es responsable de decidir si un determinado servicio se instancia localmente o solicita la instancia del servicio a otro nodo de la red (balanceo de instancias de servicios). Además de responder las solicitudes de otros nodos para obtener la imagen del contenedor de un servicio o su especificación y de presentarse a otros nodos de la red.

Para ello las instancias se almacenan en Python3, dando uso de un diccionario seguro ante concurrencia con el objeto threading.Lock de Python, con el formato descrito a continuación:



Antes de explicar cada uno de los elementos de la imágen se debe de recalcar la diferencia entre dependencia interna y externa.

- una dependencia interna es aquella que el nodo decide ejecutar localmente.
- una dependencia externa es aquella que el nodo decide solicitar a otro nodo de la red.

Donde:

- peer_ip Se refiere al solicitante (servicio o nodo) de la instancia.
- container_ip Se refiere a la dirección ip que posee el contenedor de la instancia. Se da en caso de que la dependencia sea interna.
- container_id Se refiere al identificador que posee el contenedor de la instancia. Se da en caso de que la dependencia sea interna.

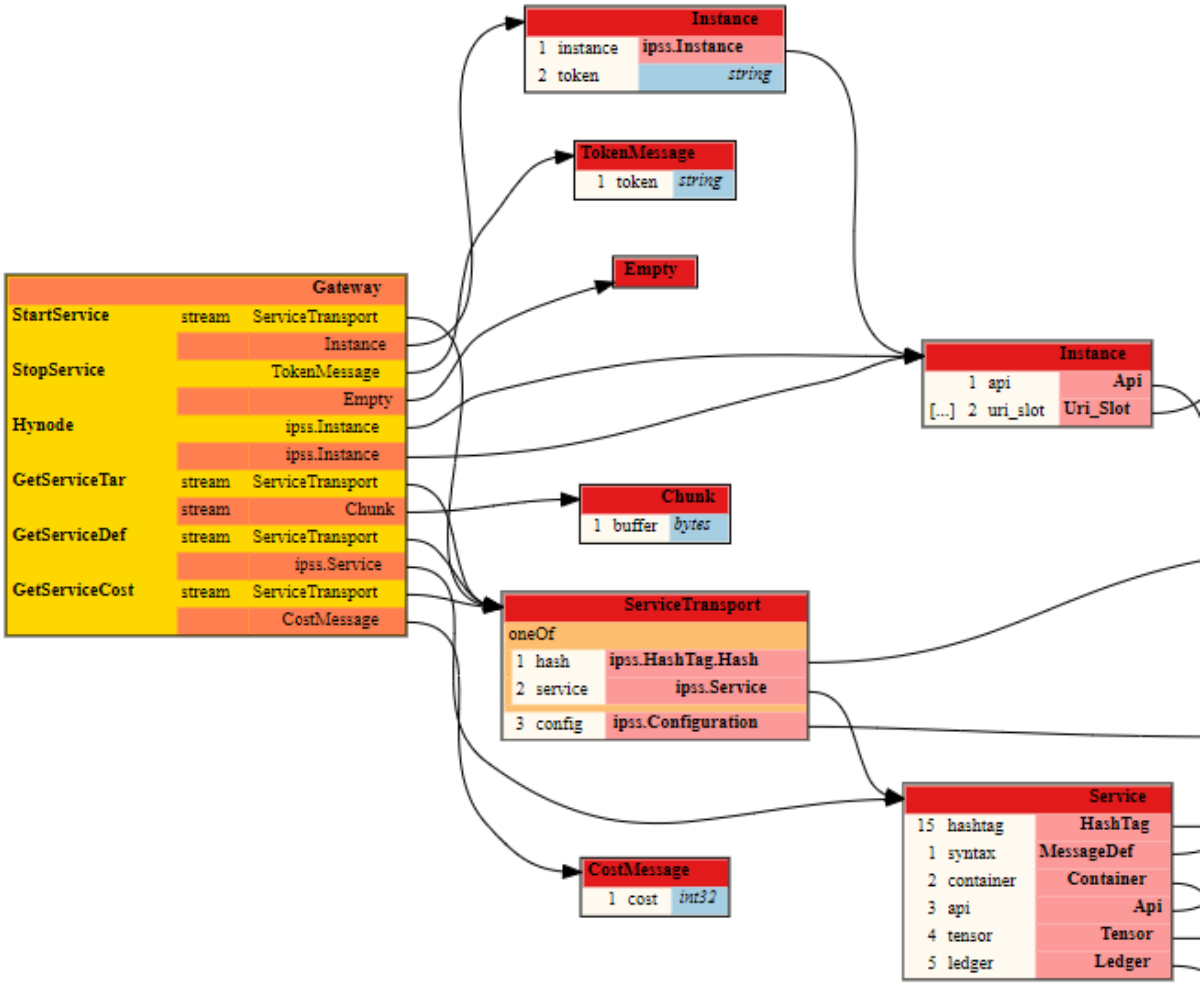
- node_ip:node_port Es la uri del nodo donde se ha solicitado la instancia. Se da en caso de que la dependencia sea externa.
- his_token Se refiere al token que posee la instancia en el nodo donde se le solicitó. Se da en caso de que la dependencia sea externa. Este formato del token varía entre unos nodos y otros, por lo que podría tener otro formato al que se describe acontinuación.

El token de una instancia posee el formato:

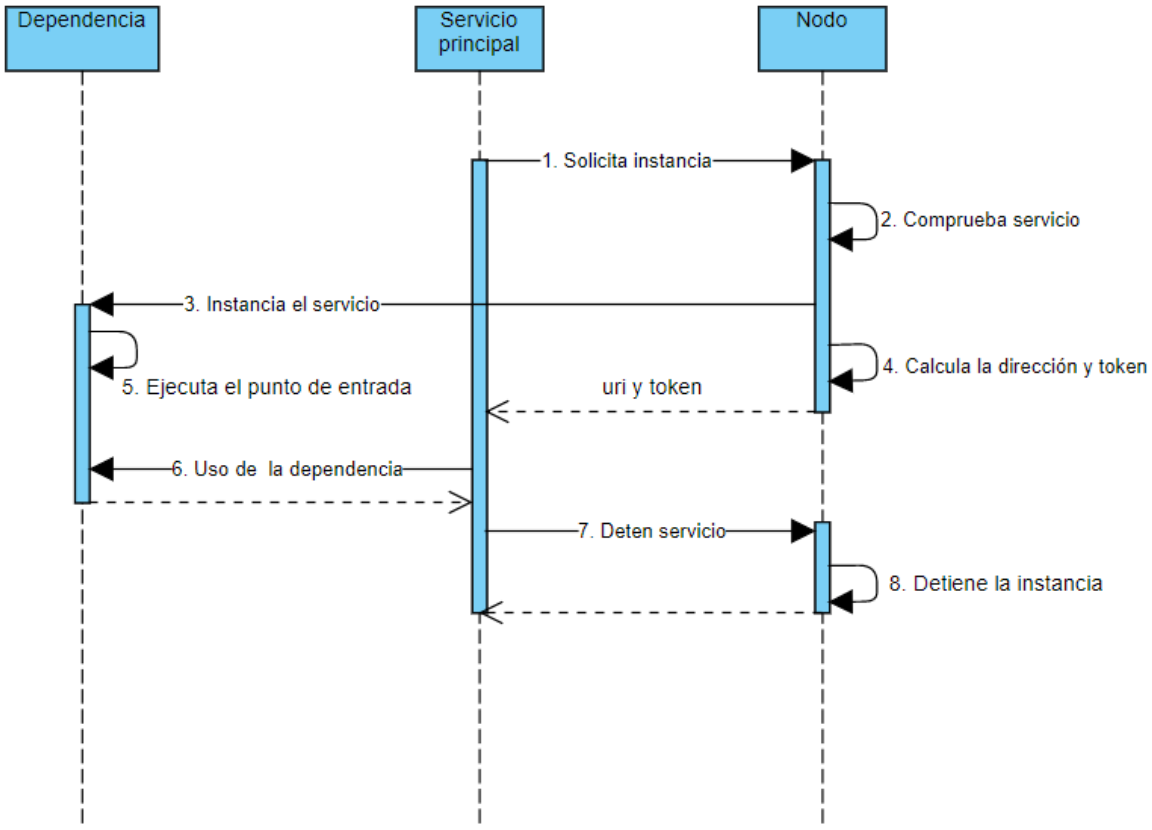
- peer_ip##container_ip##container_id en caso de una dependencia interna.
- peer_ip##node_ip:node_port##his_token en caso de una dependencia externa.

Sobre la interfaz de aplicación del servicio. Es este componente quien se comunica tanto con los servicios que instancia el nodo como con el resto de nodos de la red, para ello se da uso de los siguientes métodos mediante un servidor Grpc:

- StartService, recive la especificación de un servicio mediante 'streaming', primero recive únicamente los identificadores del servicio que posea el cliente, si comprueba que tiene el servicio comienza a instanciarlo, si no espera a que le llegue la especificación completa. Instancia el servicio especificado y retorna tanto una dirección con la que el solicitante pueda comunicarse con la instancia como un token con el que verificar al solicitante posteriormente.
- StopService, recive el token asignado a una instancia que el nodo: o bien ejecuta, o bien mando ejecutar, se encarga de detener las dependencias de esa instancia, detener su contenedor y eliminarla del registro.
- HyNode, recive la dirección de un par y le retorna la suya. El objeto ipss.Instance posee en este caso unicamente la dirección del nodo.
- GetServiceTar, recive la especificación de un servicio (igual que StartService), comprueba si posee la imagen de Docker de ese servicio, si es asi lo comprime y se lo manda en partes por streaming, en caso contrario lo puede buscar en nodos de otras redes distintas a la del solicitante.
- GetServiceDef, recive identificadores de un servicio por streaming y comprueba si posee la especificación de este. Si la posee la retorna. Si no lo posee puede buscar en nodos de redes distintas a la del solicitante, si las encuentra almacena la especificación y la retorna, si no salta excepción.
- GetServiceCost, recive la especificación de un servicio (igual que StartService), calcula y retorna el coste que tendría ejecutar localmente una instancia de este.



En el siguiente diagrama de secuencia (figura xxx) se expone el proceso de instanciar un servicio. Se parte del supuesto en que la instancia de un servicio requiere dar uso de otro servicio, por tanto solicita a su nodo que se la cree.



Los agentes que participan en el diagrama de secuencia anterior (figura) son los siguientes:

- Servicio principal, es la instancia del servicio que solicita instanciar otro a su nodo.
- Nodo, es el nodo al cual el Servicio principal solicita la generación de la nueva instancia.
- Dependencia, es la nueva instancia de la que el Servicio principal deseaba dar uso.

A continuación se exponen cada una de las interacciones durante la secuencia:

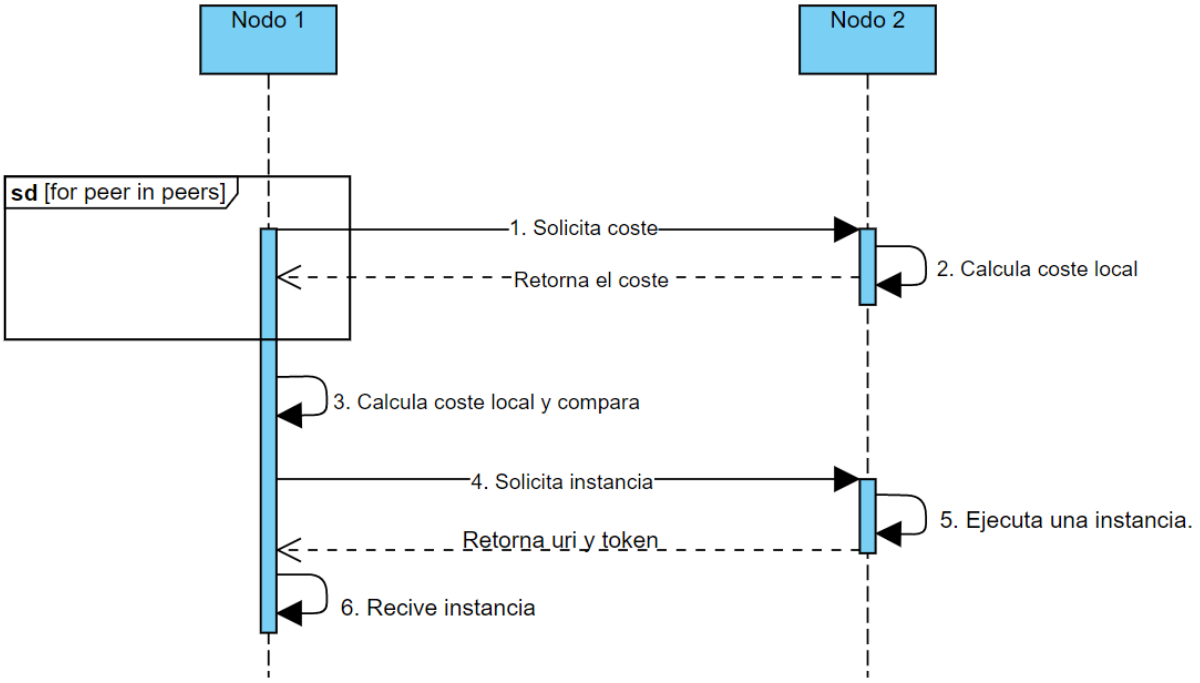
1. La instancia del servicio principal envia a su nodo la especificación de otro servicio dando uso del método StartService de este, suponiendo asi que el nodo se encargará de que se ejecute una instancia de este servicio, y espera que le devuelva la dirección de la nueva instancia, asi como el token pertinente.
2. El nodo, tras recibir la peticion del Servico principal, comprueba que posee la especificación del servicio y su imagen Docker, en caso de no tenerlo lo buscaría en la red.
3. El nodo instancia el servicio solicitado, cargando la configuración (archivo __config__ en su raiz) y ejecutando su punto de entrada.
4. La dependencia se inicia con la ejecución descrita en el Entrypoint de su especificación.
5. El nodo obtiene la dirección de la instancia usando el cliente de Docker y calcula el token correspondiente de la misma. Almacena la instancia en el registro como instancia interna y al servicio solicitante de esta como padre. Finalmente le retorna a este la dirección y token de la instancia.

6. El servicio principal (solicitante de la instancia) da uso de la dependencia mediante la dirección que le ha proporcionado el nodo.

7. El servicio principal decide detener la dependencia, por lo que ejecuta el método StopService del nodo enviandole el token de la dependencia.

8. El nodo detiene el contendor de la dependencia y la elimina del registro.

5.3.0.4 Balanceo de instancias de servicios Cuando el nodo recibe una solicitud para instanciar un servicio es este el que decidirá en que nodo de la red se debe de instanciar, localmente o en otro nodo. Para ello compara el coste de ejecución de la instancia en cada uno de los nodos que posee el registro (MongoDB) con el coste de ejecutar la instancia localmente y escoge el de menor coste. Ese mismo coste, el de ejecutar la instancia localmente, es el que retornará a otros métodos cuando le pregunten por el coste de ejecutar una instancia (método Grpc GetServiceCost). El coste de ejecutar una instancia localmente se calcula a partir de la suma del número de contenedores que se estan ejecutando en ese momento localmente junto con el coste de buscar y construir su servicio en caso de ser necesario (este coste es constante).
En el siguiente diagrama de secuencia (figura xxx) se expone el proceso de balanceo al ejecutar nuevas instancias de servicios. Se parte del supuesto en que un servicio a recibido la solicitud para ejecutar la instancia de un servicio.



Los agentes que participan en el diagrama de secuencia anterior (figura) son los siguientes:

- Servicio principal, es la instancia del servicio que solicita instanciar otro a su nodo.
- Nodo 1, es el nodo al cual el Servicio principal solicita la generación de la nueva instancia.
- Nodo 2, es otro nodo en la red.

A continuación se exponen cada una de las interacciones durante la secuencia:

1. Nodo 1 solicita el coste de ejecutar una instancia del servicio a cada uno de sus pares en la red. Da uso del método GetServiceCost enviando la especificación del servicio.

2. Nodo 2 calcula el coste de ejecutar el servicio localmente. Si no lo puede ejecutar lanzará una excepción, por el contrario le retorna el coste a Nodo 1.

3. Nodo 1 calcula el coste de ejecutar el servicio localmente y lo compara con los costes recividos. En este caso decide que el coste más bajo es el de Nodo 2.

4. Nodo 1 le solicita Nodo 2 que ejecute una instancia del servicio mediante el método StartService, le envia de nuevo la especificación de este, y espera que le retorne su dirección y token.

5. Nodo 2 ejecuta la instancia del servicio que le entrega Nodo 1, guarda la instancia en el registro considerando que el padre es Nodo 1 y le retorna su dirección y token.

6. Nodo 1 recive la dirección y token de Nodo 2, guarda la instancia en el registro como externa.

5.3.0.5 Distribución de contenedores Con el fin de atender las solicitudes de obtención de un contenedor el nodo proporciona un método Grpc denominado GetServiceTar. Éste recive la especificación de un servicio y, si puede proporcionarlo, comprime el contenedor y envía el archivo en formato .tar dividiendolo en trozos y enviandolos en streaming, en caso de no poder proporconar el contenedor salta una excepción que recive el cliente.
Para la busqueda de contenedores, el nodo debe de iterar entre sus pares (resto de nodos de la red) y comprobar si alguno posee el contenedor dando uso del método anteriormente definido.

Para la distribución de especificaciones de servicios se utilizan métodos Grpc con mecánicas similares.

5.3.0.6 Conexión con nuevos nodos Al añadir un nuevo nodo a la red se debe de usar el comando 'python3 zeroconf.py <uri del nuevo nodo>' desde la consola de un nodo, el comando ejecutará el método Grpc HyNode en el supuesto nuevo nodo, si este le responde con un objeto Instance, el nodo le retornará su instancia y lo tomará como un nodo completamente operativo, si no responde olvidará la solicitud.

5.4 Fase de desarrollo de servicios

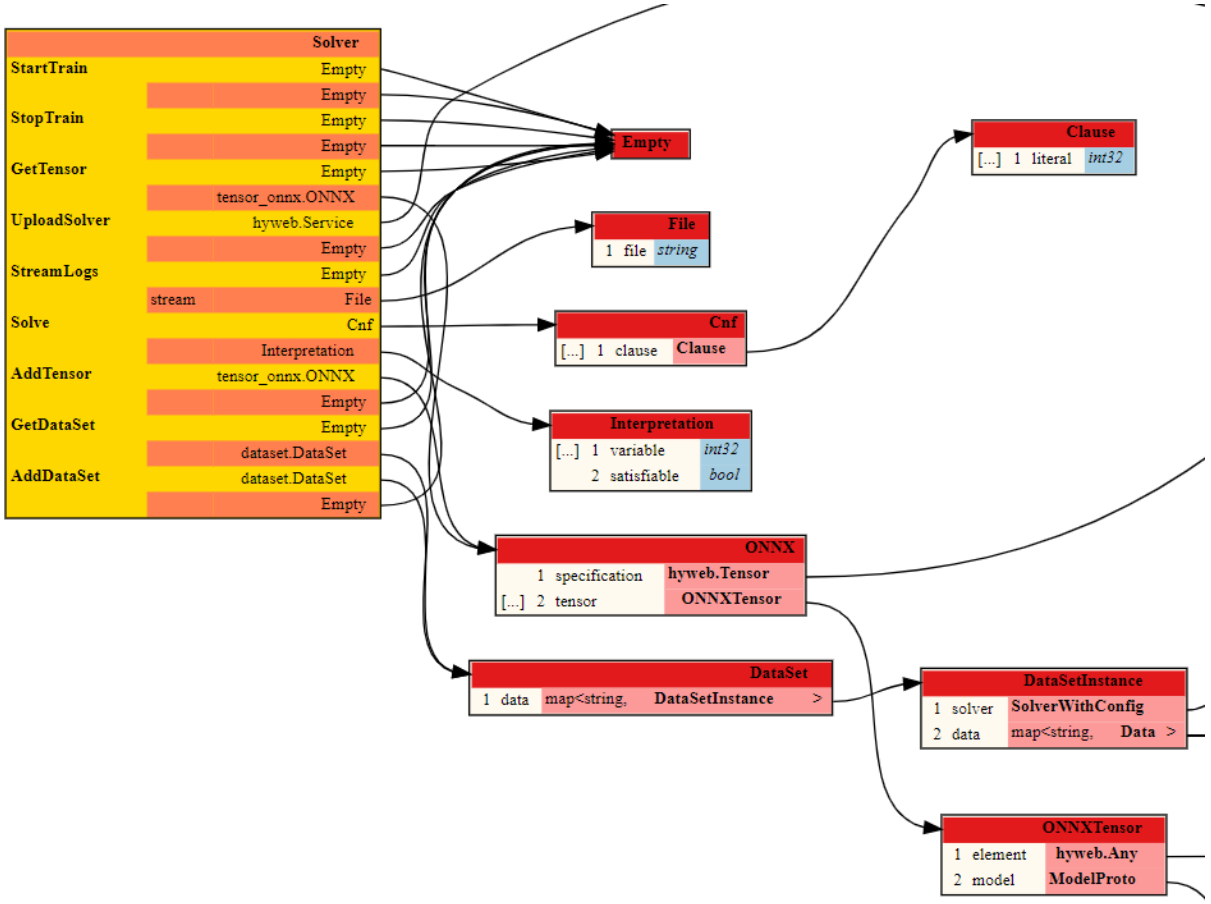
Esta fase consiste principalmente en desarrollar el servicio de clasificación de solvers, este tiene la finalidad de: por un lado atender solicitudes para resolver CNF's, y por otro realizar un entrenamiento con cnf's aleatorios para clasificar diferentes servicios SATsolvers, esto con la intencion de utilizar el SATsorter más adecuado para cada CNF durante las solicitudes reales.

5.4.0.1 Servicio de clasificación El servicio realiza varias tareas de manera simultanea, al igual que el nodo, se ha optado por dar uso de Python3 como tecnología sobre la que desarrollar su lógica. El punto de entrada del servicio es el fichero start.py, donde se inicia el servidor Grpc tras acceder a la configuración inicial del servicio (archivo __config__ proporcionado por el nodo), tomar las variables de entorno e iniciar diferentes procesos secundarios.
Da uso de un servicio con el que obtener cnfs random, por lo que solicita una instancia de este en cada sesión de entrenamiento y la mata al finalizar la sesión. Además de esta dependencia también se le añade cada uno de los solvers, los cuales pueden ser añadidos por el cliente a través del servidor

Grpc. El servicio de obtención de cnfs random no ha sido desarrollado durante el proyecto, solo adaptado, se da uso de un código del profesor Josep Argelich, por lo que su lógica no será desarrollada en la memoria, retorna un cnf de cualquier tipo, satisfactible o insatisfactible.
También da uso de un servicio con el que obtener un modelo de regresión lineal a partir de un set de datos. Este se desarrolla más adelante.

El servidor Grpc posee una serie de métodos con los que atender las solicitudes del cliente, estos son:

- StartTrain, inicia una sesion de entrenamiento si no está ya inicializada.
- StopTrain, si existe, detiene la sesión de entrenamiento.
- UploadSolver, recive la especificación de un servicio de resolución de SAT y la añade al sistema.
- StreamLogs, retorna los 'logs' del sistema mediante streaming.
- Solve, recive un cnf, calcula cual es el mejor solver con el que resolverlo y lo resuelve dando uso del mismo. Finalmente retorna el resultado.
- GetTensor, recive el modelo de regresión lineal entrenado.
- AddTensor, permite añadir un modelo para combinarlo con el que se encuentra ya en el servicio. Este método no esta implementado.
- GetDataSet, obtiene el set de datos que se han obtenido del entrenamiento utilizados para entrenar el modelo.
- AddDataSet, recibe un nuevo set de datos y lo combina con el que ya tiene el servicio.



Las partes principales en las que se divide la lógica del servicio son:

Interfaz de regresión Ubicado en el archivo regresion.py. Durante el inicio del proceso principal (start.py) se obtiene el objeto regresion.Session, el cual sigue el patrón Singleton, este objeto es usado durante toda la vida del servicio. Se encarga de instanciar y proporcionar los métodos con los que dar uso del servicio que realiza la regresión lineal y mantenerlo, esto es: control de errores y solicitud de creación y suspensión de instancias al nodo, a cierto número de errores destruye la instancia y solicita otra. También posee el set de datos principal, un método para añadirle tanto los datos generados por el entrenamiento como los que pueda proporcionar el cliente, asi como último modelo de regresión.
Posee una función de mantenimiento del servicio encargada de ordenar ejecutar la regresión y guardar el modelo en el servicio de clasificación cada cierto periodo de tiempo, esta se ejecuta en el hilo de mantenimiento.
Para realizar la union de dos datasets la formula queda:

$$\frac{i * \overline{S_i} + j * \overline{S_j}}{i+1} = \overline{S_{i+j}} \tag{3}$$

Sistema de entrenamiento Ubicado en el archivo train.py. Durante el inicio del proceso principal (start.py) se obtiene el objeto trainer.Session, el cual sigue el patrón Singleton, este objeto es usado durante toda la vida del servicio.
El objeto train.Session se ocupa de:

- Gestionar el uso del servicio de obtención de cnfs random.
- ejecutar la sesión de entrenamiento.
- detener la sesión de entrenamiento.
- Añadir nuevos datos.

Cuando se inicia una sesión de entrenamiento (método StartTrain) desde el hilo principal se crea un hilo que ejecuta el método init, este inicia un servicio de cnfs random y, en cada iteración, genera un cnf y lo prueba en cada uno de los solvers. Solo se detiene con la señal de la variable do_stop generado desde el hilo principal por el método StopTrain.
Sobre el sistema de puntuación de los solvers, la puntuación que un solver recibe de la resolución de un cnf es inversamente proporcional al tiempo que el solver tarda en responder. En caso de que tarde en responder demasiado (existe un 'timeout' aplicado en la llamada Grpc de la variable de entorno TRAIN_SOLVERS_TIMEOUT, 30 segundos por defecto) se puntua en negativo (-1/time_out). En caso de que devuelva una interpretación se comprueba que esta sí satisface el cnf (operación con coste polinomial), si lo satisface puntua en positivo (+1/time) y registra que se demostro la satisfactibilidad del cnf, si no la satisface puntua en negativo (-1/time). En caso de que responda que el cnf es insatisfactible se guarda en una lista que ese solver tardo x tiempo en decir que era insatisfactible, al terminar de probar todos los solvers con ese cnf se comprueba si hubo alguno que demostrara su satisfactibilidad, de no ser asi, los solvers que lo dieron como insatisfactible puntuan en positivo (+1/time) y si se demostró la satisfactibilidad puntuan en negativo (-1/time).

El incremento de la puntuación para un cnf concreto de un solver determinado es:

$$\frac{\sum_{i=1}^{i} S + S_{i+1}}{i+1} = \overline{S_{i+1}} \Longrightarrow \frac{i * \overline{S_i} + S_{i+1}}{i+1} = \overline{S_{i+1}} \tag{4}$$

Cada cierto número de iteraciones en el entrenamiento (SAVE_TRAIN_DATA: 10 por defecto si la variable de entorno no indica lo contrario) añade el set de datos al set de datos principal que posee la interfaz de regresión, y elimina los datos generados durante la iteración. No resetear los datos cada vez que se añaden al set principal provocaría desajustar la media de cada punto dando más importancia a los más antiguos.

Sobre la gestión del servicio de generación de cnfs random, se crea una instancia al iniciar cada sesión de entrenamiento y se destruye al finalizarla. Además de eso es necesario solicitar una nueva instancia en caso de que la instancia actual del servicio acumule una cantidad determinada de errores (CONNECTION_ERRORS: por defecto 5), originados al solicitar un nuevo cnf ya sea por un error de Grpc o porque salta su 'timeout'.

Sobre el mecanismo para detener el entrenamiento, hay que tener en cuenta que para evitar que el método stop, usada por el método StopTrain del servidor Grpc en start.py, mate el hilo durante un entrenamiento el método stop emite el mensaje de que se debe de parar el entrenamiento en la siguiente vuelta, y a continuación espera a que el entrenamiento salga del bucle y termine. De

esta forma se asegura que no deja instancias fuera de pila y por tanto instancias zombie (la pila se aborda en la tercera fase de desarrollo).

Administrador de solvers Ubicado en el archivo _solve.py. Durante el inicio del proceso principal (start.py) se obtiene el objeto solve.Session, el cual sigue el patrón Singleton, este objeto es usado durante toda la vida del servicio.
El objeto solve.Session se ocupa de:

- gestionar las instancias de todos los solvers
- solicitar a un solver determinado que resuelva un cnf determinado
- balancear las solicitudes entre las distintas instancias de un mismo solver en caso de ser necesario (este punto se aborda durante la tercera fase de desarrollo).

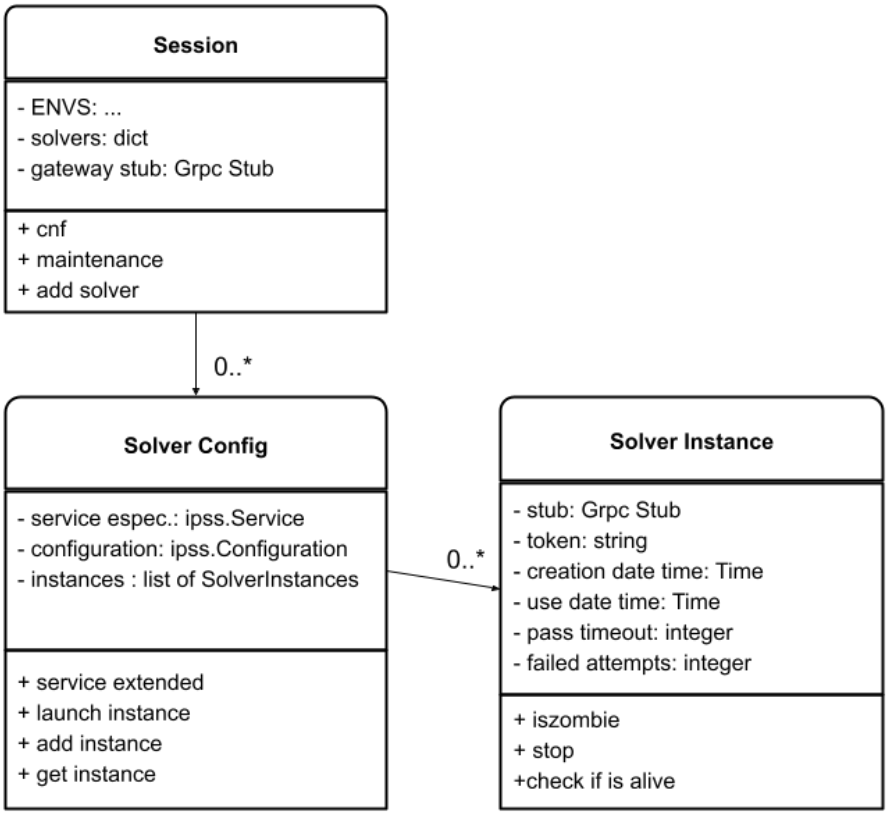
Posee una funcion dedicada a controlar el estado de las instancias de los solvers, ejecutada en el hilo de mantenimiento. Comprueba si las instancias siguen activas resolviendo un cnf básico ([[1,]]), en caso de obtener un error se encarga de destruir la instancia.

El objeto solve.Session también posee un método con el que añadir nuevos solvers a su registro, estos se almacenan en un diccionario del tipo identificador:SolverConfig. El objeto SolverConfig detalla la definición de un servicio junto con una configuración del mismo determinada.

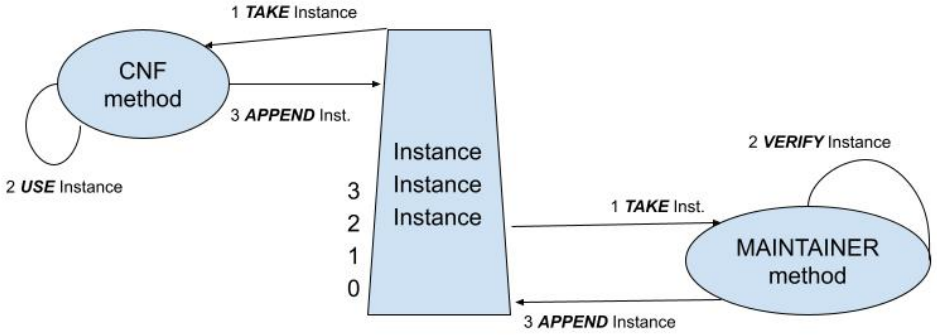
Finalmente, el objeto solve.Session permite resolver cnfs dado un cnf, el identificador de un solver con una configuración determinada, y con la posibilidad de especificar el timeout de la operación y proporcionar la especificación del solver junto con su configuración, por si con el identificador no se encuentra. En caso de que ese solver se encontrara en el registro, se resuelve el cnf y se retorna la respuesta junto con el tiempo que tomó resolverla. En caso de que no se encuentre y se halla proporcionado una especificación la añadirá al registro y retornará error.

Sobre el balanceo de solicitudes. En el supuesto de que el cliente realiza, al mismo tiempo, multitud de solicitudes para resolver solvers, podría darse el caso de que, en un mismo instante de tiempo, se requiera dar uso de un solver en concreto. El solver en cuestion podría ser capaz de resolver varios cnfs simultaneamente, dentro del mismo contenedor, aunque de esa forma no se aprobechan las ventajas del sistema distribuido. Con el objetivo de que un varios cnfs se resuelvan simultaneamente por un mismo solver el clasificador crea distintas instancias de este cuando es necesario, de esta forma el nodo se encargará de instanciar el servicio entre los distintos nodos de la red.

Antes de entrar en detalle sobre la estructura sobre la que se manejan las instancias se debe de exponer el diagrama de clases siguiente:



Como se puede observar en el diagrama anterior, para manejar las instancias de cada solver, se adopta una estructura de un diccionario de solvers (<solver config. id>:< objeto SolverConfig>), donde cada objeto SolverConfig posee una lista de instancias (objeto SolverInstance). Cuando se requiere la instancia de un solver determinado se quita de la lista y cuando se deja de usar se vuelve a introducir en la lista. El acceso a todo el diccionario se proteje contra la concurrencia mediante el uso de un objeto threading.Lock. El identificador de un objeto SolverConfig se obtiene mediante el algoritmo hash sha3-256 de su binario Protobuf.
En el siguente esquema se expone el uso de la pila de instancias de un único solver:



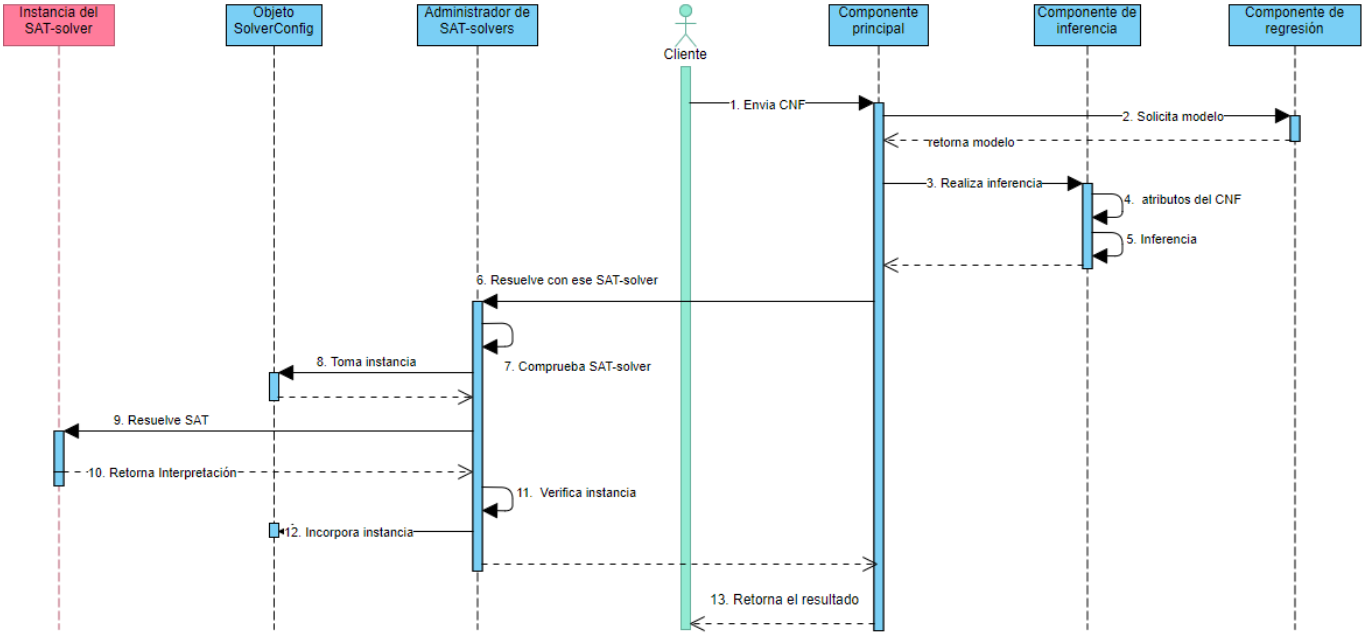
Hay dos métodos que obtienen e introducen instancias en las listas, resolver un cnf (Session.cnf) y el del hilo de mantenimiento (Session.mainteance).
El método para resolver un cnf da uso de la lista como si se tratara de una pila normal, es decir, toma el elemento que se encuentra en el índice más elevado de la lista y coloca el elemento en el índice más elevado de la lista.
El método de mantenimiento da uso de la lista como si se tratara de una pila inversa, es decir, toma el elemento que se encuentra en la primera posición de la lista y coloca el elemento en la primera posición también.
El motivo de adoptar esta configuración es que las instancias tienen contadores sobre el último tiempo en que se usaron, estos solo se actualizan al solicitar resolver cnfs (el método Session.cnf), por lo que cuando el mantenedor de instancias la toma comprueba si se paso de tiempo en deshuso, si es asi no lo vuelve a cargar en la lista, de lo contrario la deja en primera posición. De esta forma se sabe seguro que las instancias en posiciones superiores llevan menos tiempo desde su último uso.

Si se toma una instancia, se debe de asegurar que al terminar de usarla, o bien se agrega a su lista correspondiente, o bien se solicita al nodo detenerla. No asegurar esto ocasiona un bug importante ya que la instancia quedaría en estado zombie hasta que el clasificador fuera eliminado (en el supuesto en que el nodo donde se instancio matara dependencias que fueran a quedar zombies).

Sistema de inferencia Ubicado en el archivo _get.py. Trata de seleccionar el solver que mejor rendimiento va a dar para un cnf determinado. Realiza inferencia sobre el modelo de regresión lineal

de cada uno de los solvers contenidos en él.
La inferencia es el proceso por el cual se derivan conclusiones a partir de premisas, es decir, a partir del modelo obtenido mediante regresión lineal se supone la puntuación o eficiencia que se obtendría para un cnf determinado con un SAT-solver concreto. Para realizar la inferencia se da uso de la libreria ONNX-Runtime de Microsoft.

A continuación se expone un diagrama de secuencia (figura xxx) sobre el supuesto en el que un cliente solicita resolver SAT al servicio de clasificación, teniendo este el modelo en memoria y el solver del que dar uso instanciado.



Los agentes que participan en el diagrama de secuencia anterior (figura) son los siguientes:

- Cliente que solicita al servicio la resolución de un problema SAT.
- Componente principal del servicio, es decir, el servidor Grpc.
- Componente de inferencia del servicio, donde se selecciona el SAT-solver dando uso del modelo entrenado.
- Componente de regresion del servico, que, aparte de comunicarse con el servcio de regresión, posee el último modelo entrenado.
- Administrador de SAT-solvers, componente del servicio que gestiona las instancias disponibles de cada SAT-solver.
- Objeto SolverConfig, perteneciente al administrador de SAT-solvers, controla las instancias de un SAT-solver concreto con una configuración determinada.
- Instancia del SAT-solver, instancia del servicio SAT-solver del que el sistema de inferencia decide dar uso para resolver ese problema concreto.

A continuación se exponen cada una de las interacciones durante la secuencia:

1. El cliente solicita al servicio, mediante el método Grpc Solve, que le resuelva un CNF determinado.

2. El componente principal del servicio obtiene el modelo del componente de regresión, quien poseía el último modelo entrenado por el servicio de regresión.

3. El componente principal del servcio solicita al componente de inferencia la especificación del SAT-solver más efectivo a partir del CNF y el modelo proporcionado previamente por el cliente por el componente de regresión respectivamente.

4. El componente de inferencia obtiene los parámetros del CNF, estos son: el número de clausulas y el número de literales que posee.

5. El componente de inferencia obtiene el SAT-solver con el supuesto mejor rendimiento para ese número de clausulas y literales.

6. El componente principal solicita resolver el CNF proporcionado con el SAT-solver propuesto al administrador de SAT-solvers.

7. El administrador de SAT-solvers verifica poseer ese SAT-solver con esa configuración concreta en su registro. Lo tiene. En caso contrario lo añadiría y saltaría una excepción.

8. Se adquiere una de las instancias del SAT-solver de la pila del objeto pertinente a ese SAT-solver, de esta forma otra solicitud no conocerá de la existencia de esa instancia al no encontrarla en la pila. En este caso si hay instancias en pila, de no haberlas, lanzaría una nueva mediante el proceso expuesto en el diagrama de la figura xxx.

9. Desde el administrador de SAT-solvers se solicita la resolución de SAT a la instancia proporcionada enviándole el CNF.

10. El SAT-solver retorna una interpretación o dice que es insatisfactible siempre y cuando no sale el tiempo de espera por parte del servicio principal.

11. Verifica la disponibilidad de la instancia. Si hubiera dado algun problema en el paso anterior le pediría resolver un CNF simple para comprobarlo, en caso de fallar si que pediría la detención de la instancia.

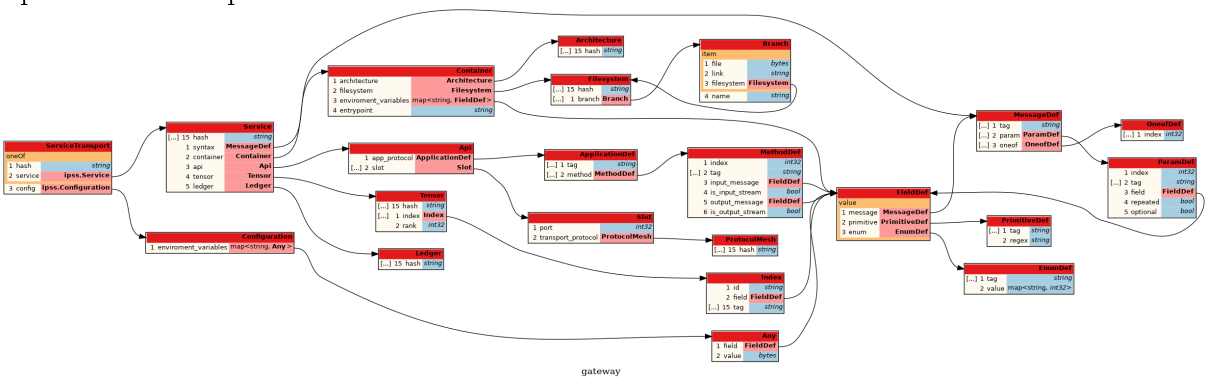
12. Tras haber dado uso de la instancia y verificar que esta se encuentra activa, la incorpora de nuevo a la pila, de forma que esta disponible para usarse por el entrenamiento u otras solicitudes similares a esta.

13. El administrador de SAT-solvers retorna el resultado al componente principal, este a su vez lo retorna al cliente.

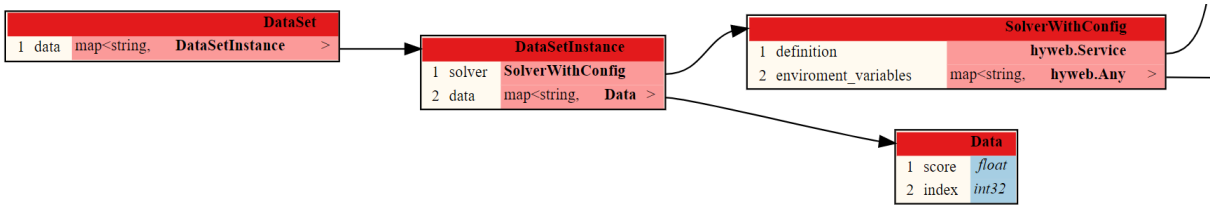
Hilo de mantenimiento Se trata de un hilo independiente iniciado desde start.py antes de ejecutar el servidor Grpc. Ejecuta tanto el método de mantenimiento del administrador de solvers como del de regresión cada cierto periodo de tiempo (MAINTENANCE_SLEEP_TIME: 100 segundos por defecto) para el administrador de solvers y (TIME_FOR_EACH_REGRESION_LOOP: 900 segundos por defecto) para el de regresión.

El patrón singleton o instancia única es un patrón de diseño que permite restringir la creación de objetos pertenecientes a una clase o el valor de un tipo a un único objeto. De esta forma se limita a que solo pueda haber una instancia de las clases train.Session, regresion.Session y solve.Session.

Para enviar la especificación de un servicio por Grpc se utiliza el mensaje Grpc ServiceTransport, este optimiza el proceso enviando por streaming cada hash de la especificación (una por una) y finalmente la especificación completa, además de enviar la configuración (en caso de usarse para solicitar una instancia) en tan solo una de las partes. Cuando el servidor recive el stream comprueba con las hashes si ya posee el servicio, si lo posee para el stream y no es necesario enviar la especificación completa.



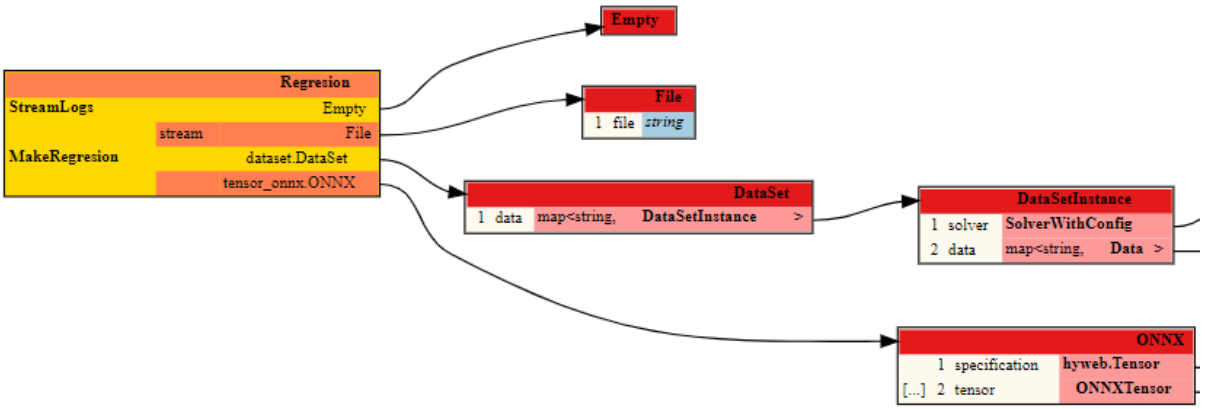
El set de datos alamcena la información generada durante el entrenamiento o la entregada al servicio de clasificación desde fuera, este almacena la puntuación de cada SAT-solver por cada tipo de CNF provado. Y se utiliza el mismo modelo tanto para intercambiar datos entre el servicio de clasificación y regresión como para almacenar los datos en cada uno de los dos servicios.



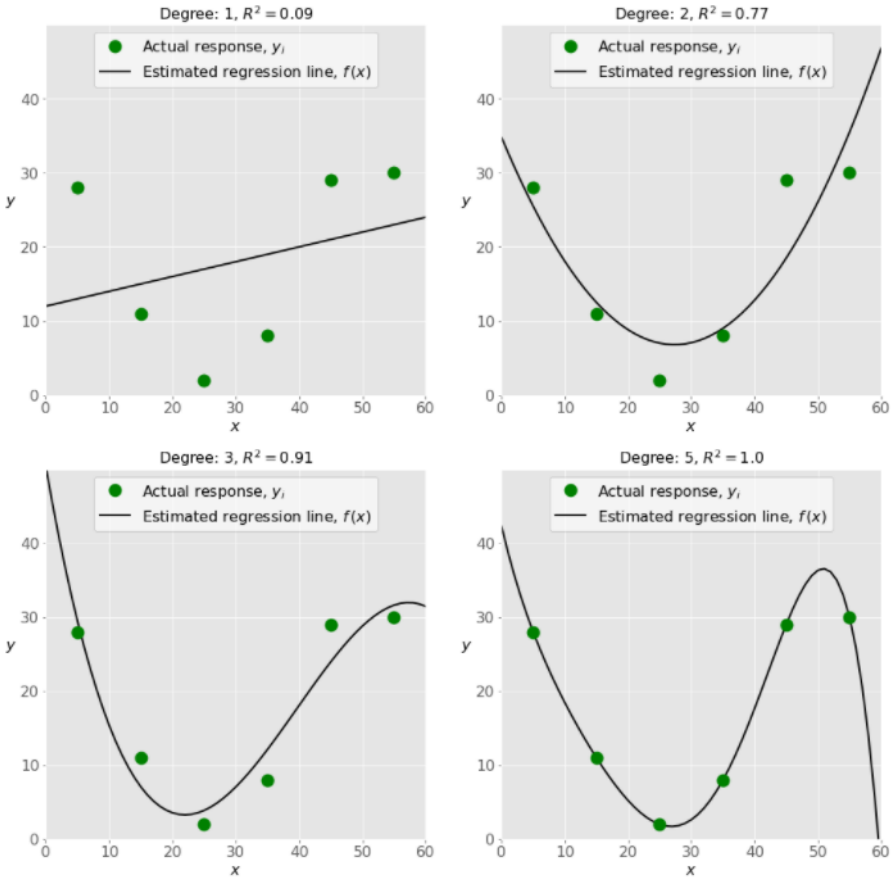
5.4.0.2 Servicio de regresión Sobre el servicio de regresión, este se ocupa parte de la tarea del servicio de clasificación, el cual lo instancia. Este recive sets de datos, los combina y cada cierto periodo de tiempo obtiene un modelo a partir del set de datos principal. Se compone de tres partes:

Servidor Grpc Ubicado en el archivo start.py. El servidor Grpc posee una serie de métodos con los que atender las solicitudes del cliente, estos son:

- StreamLogs, retorna los 'logs' del sistema mediante streaming.
- MakeRegresion, recive un set de datos, realiza la regresion lineal de este y retorna el modelo.



Función de regresión Ubicado en el archivo regresion.py La regresión de cada solver consiste en realidad en una regresión por cada grado hasta un máximo estipulado (MAX_DEGREE: 100 por defecto si en la variable de entorno no se indica lo contrario) y escoger aquella función que se ajuste más a los datos, esto con el fin de maximizar el ajuste en la función del modelo sobre los datos. De esta forma se realizan un total de <número de solvers> * MAX_DEGREE regresiones cada TIME_FOR_EACH_REGRESSION_LOOP segundos.
El modelo posee el formato de una lista de modelos de ONNX (Intercambio de Redes Neuronales Abierto), un ecosistema de inteligencia artificial de código abierto, asociados cada uno a un SAT solver (especificación del servicio). Se da uso del estandar Onnx por ser un estandar abierto para compartir modelos de redes neuronales entre distintas librerías de inteligencia artificial (TensorFlow, PyCharm, Sklearn . . .), aunque en este caso se utiliza para compartir modelos de regresión lineal, permite que otro servicio de clasificación continue el modelo entrenado por el clasificador sin tener que utilizar la librería de la él da uso (Sklearn).

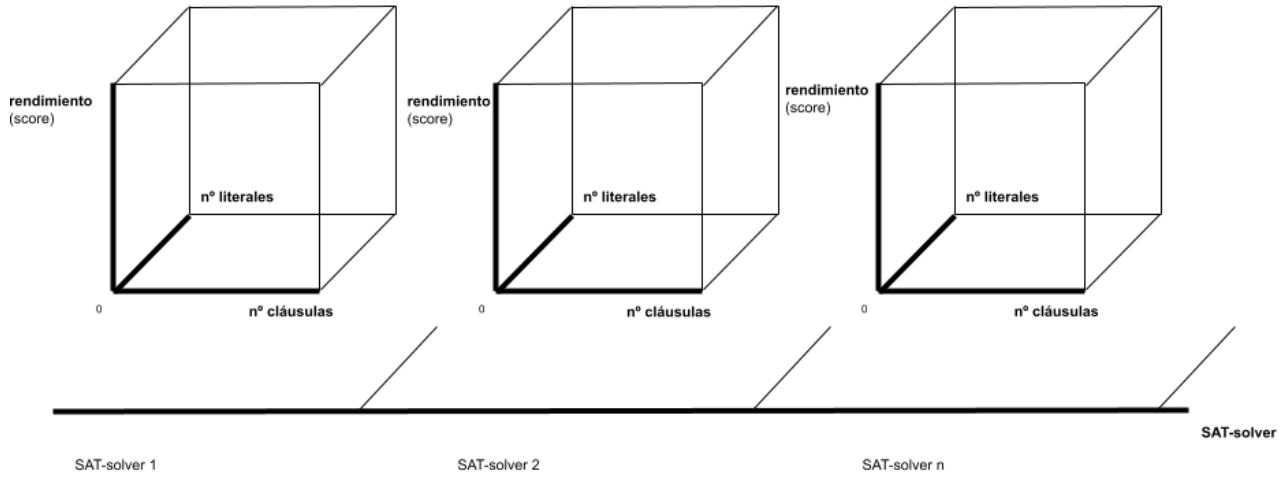


Se toma la función con mayor ajuste sobre los datos.

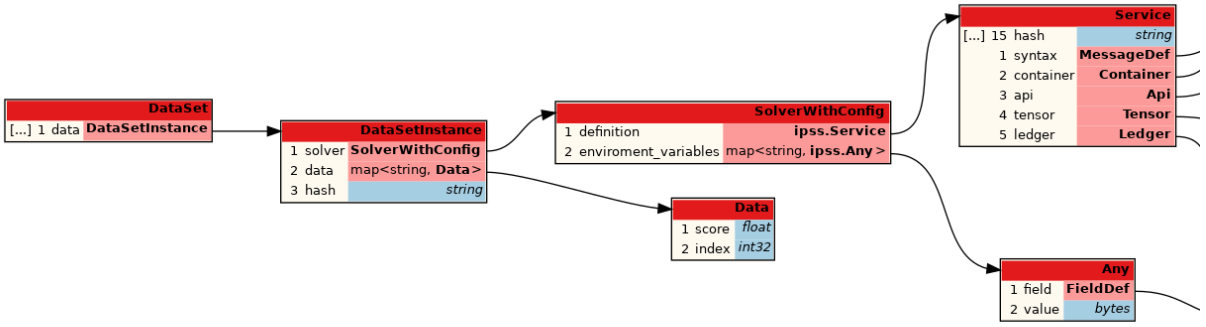
El modelo, por tanto, posee 4 dimensiones independientes:

- El número de clausulas del CNF.
- El número de literales del CNF.
- El rendimiento ofrecido, proporcional a la capacidad de acierto e inversamente proporcional al tiempo de respuesta.
- La especificación del algoritmo SAT-solver con una configuración determinada. Única dimensión de valor discreto.

De forma que se puede deducir el rendimiento ofrecido por un tipo de CNF concreto en un SAT-solver concreto.



El formato del set de datos consiste en una puntuación por cada tipo de cnf (número de instancias * número de clausulas) junto con el número de entradas de cada puntuación por cada solver con una configuración determinada, su especificación se encuentra también en el set de datos.



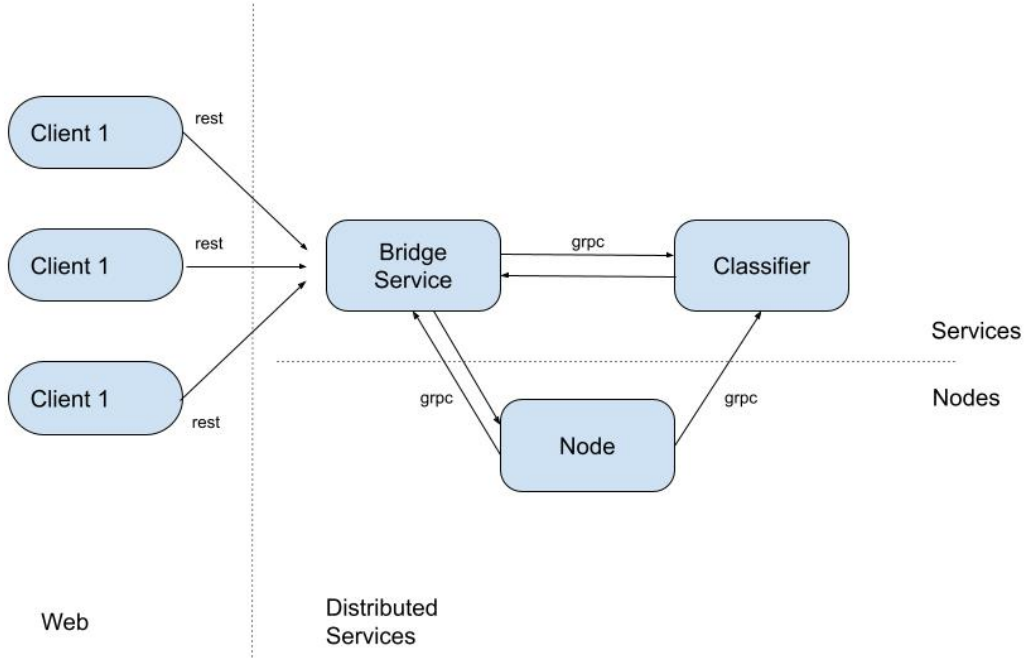
La decisión de ejecutar la regresión del modelo en un servicio independiente viene motivada por que, tras realizar pruebas sobre el sistema completo (clasificación y regresión en un mismo servicio), se observó una disminución del rendimiento al ejecutar cada iteración de regresión. Por tanto se decidió separar la función de regresión sobre el servicio de clasificación con el fin de mejorar la eficiencia del entrenamiento y resolución de SAT cuando se está realizando la regresión del set de datos. Principalmente esto permite que el peso del clasificador se balancee, ejecutar la regresión en otro servicio da la posibilidad de que esto ocurra en un nodo distinto si el sistema de balanceo de nodos lo considera conveniente.
Por tanto, el punto positivo de separar las funciones de un servico en varios es que se vuelven más livianos y, por tanto, da la posibilidad a los nodos para distribuir mejor su ejecución. El punto negativo es que se requiere ejecutar más procesos, dos más en este caso, para el servidor del nuevo servicio y para que mantenga la regresión, además de que se está, en última instancia, almacenando el set de datos dos veces en vez de una, debido a que la neturaleza de los servicios les origina tal

aislamiento que imposibilita que compartan memoria (esto último puede ser un punto de estudio futuro como parte de la extension de la especificación de un servicio).

6 Problemas encontrados y decisiones técnicas tomadas

6.1 ¿Cómo realizan múltiples clientes solicitudes al servicio de clasificación?

Basándose en la arquitectura anteriormente desarrollada, si un cliente quiere dar uso de un servicio, éste solicita la creación de una instancia del mismo a su nodo, por lo tanto, si n clientes quieren dar uso del servicio, serán necesarias n instancias de éste, a fin de cuentas, la instancia de un servicio, un contenedor, no es más que un hilo aislado. Por lo tanto, el paradigma cliente/servidor se ve alterado al hablar de servicios distribuidos. En vez de un servidor atendiendo múltiples clientes, un cliente solicita el despliegue de un servicio. En términos de rendimiento no debería existir una gran diferencia entre un esquema y otro, puesto que el servidor centralizado termina ejecutando n hilos para n solicitudes de clientes, en los servicios distribuidos estos hilos se aislan (esto puede ser un punto negativo sobre el uso de memoria compartida) y se reparten entre los nodos de la red.
Ahora bien, en caso de querer realizar un servicio web, al que poder, tal vez, conectar con un frontend en un navegador, será necesario un nuevo servicio el cual se encargue de recibir las solicitudes de los clientes y desplegar y dar uso del servicio de clasificación. Este servicio por tanto, puede ser comprendido como un puente entre la "web tradicional" y una red de servicios distribuidos.



6.2 Varias instancias no pueden compartir dependencias

La naturaleza que adopta la red crea una jerarquía de instancias de servicios repartidas entre los nodos de la mísma.
Esto significa que, cuando una instancia (padre) solicita una instancia de otro servicio (hijo), este último pasará a ser dependiente del primero, adoptando el padre el derecho a decidir cuando debe de morir el hijo y la responsabilidad sobre su consumo de recursos (en caso en que un nodo realice un seguimiento del consumo de cada instancia, deberá tener en cuenta el de sus dependencias). De esta forma, si se mata al padre, el nodo donde se este ejecutando su hijo podrá matarlo.
Si no fuera asi, un nodo no sabría que instancias quedan en estado 'zombie' ya que ninguno de sus clientes (instancias padre) se haría responsable.

6.3 Json vs GRPC

Json se ha convertido en prácticamente un estándar a la hora de compartir información en la web, debido a que es el sistema de serialización de Javascript, y éste, a su vez, es el lenguaje estandarizado para ejecutarse en los navegadores web.

Pero en una red de microservicios, donde los entornos de ejecución son virtualizados, Javascript pierde protagonismo y, por tanto, Json también. Es por esto que cabe preguntarse cual es la mejor forma de serializar las comunicaciones entre servicios. Al ser evidente que no existe un protocolo capaz de liderar ni en cualquier ámbito ni indefinidamente, cada Slot de un servicio definirá el protocolo soportado, esto con el fin de garantizar la escalabilidad de la arquitectura del sistema.

Concretamente para este proyecto se dará uso de Grpc, principalmente por dos razones:

- Protobuf, su lenguaje de serialización, consigue comprimir bytes de manera muy eficiente enfocando los buffers a cada topología de mensajes.
- Utiliza Http/2, por lo que está orientado a realizar streaming bidireccional.

Otros protocolos también a tener en cuenta podrían ser Flatbuffers o Capt'n proto.

6.4 ¿Porque contener todo el sistema de ficheros en la especificación de un servicio?

La especificación de un servicio debe de definirlo de manera completa para evitar redundancia entre distintos servicios. Aunque puede acarrear problemas como el tamaño de la especificación, se puede resolver en una capa de abstracción más alta, es decir, es posible implementar, por ejemplo, la compresión de la especificación de una cierta manera.

6.5 ¿Porque no añadir el Dockerfile en la especificación de un servicio?

Igual que el problema sobre el tamaño del sistema de ficheros, especificar como se debe de construir el servicio debe de ser tratado en una capa de abstracción superior a la especificación del servicio.

6.6 Identificación del algoritmo hash

Para evitar colisiones al identificar diferentes algoritmos hash estos son identificados aplicando su propio algoritmo a una entrada vacía, es decir, una cadena de bits vacía. Por ejemplo:

- sha256 / sha2-256: e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
- sha3-256: a7ffc6f8bf1ed76651c14756a061d662f580ff4de43b49fa82d80a4b80f8434a
- shake-256 of 200bits: 46b9dd2b0ba88d13233b3feb743eeb243fcd52ea62b81b82b50c27646ed5762f

De esta forma el formato es <id hash>:<hash value>. Por lo que la palabra "nodo" sería:

- sha256 / sha2-256:
  - type: e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
  - value: 755c967353885b8b5096b5492b317fa25f77384e7c57afa05748f2238a9141d9
- sha3-256:
  - type: a7ffc6f8bf1ed76651c14756a061d662f580ff4de43b49fa82d80a4b80f8434a
  - value: 2837075229aa23bf8a4139715ac8369a62c2beb0e868df5bd32a2ca6d8e56d2a
- shake-256 of 200bits:
  - type: 46b9dd2b0ba88d13233b3feb743eeb243fcd52ea62b81b82b50c27646ed5762f
  - value: 77b532f14d889b55c5e1c5f309ceb43cec543be21a0372fa56fc09afb4bb594a

De esta forma no se requiere centralizar un registro para acordar que identificador corresponde a un algoritmo concreto.

7 Resultados

Resultados de los tests realizados sobre el sistema.

8 Conclusiones

9 Trabajo futuro

En el proyecto existen varios puntos de continuación distintos.

Por un lado, ampliar la especificación de un servicio permitiría explorar nuevos tipos de aplicaciones. Por ejemplo especificar el modelo que ofrece un servicio daría el paso a un tipo de servicio que no clasifica pero si selecciona, solicitando un tipo de modelo al nodo y este retornandole direcciones de servicios que puedan proporcionarselo, o simplemente permitiría que distintos servicios que entrenan un tipo concreto de modelo lo intercambiasen.

Otro punto de estudio sobre la especificación de un servicio puede ser la posibilidad de que exista un coste por usarlo, es decir, que se especifique de alguna forma el coste que tiene usar el servicio o los requisitos a cumplir para usarlo, y que dicho servicio pueda verificar que se cubrió el costo

o cumplieron los requisitos sin necesitar confiar en el cliente. Esto podría ser funcional tambien para la interacción entre nodos a la hora de compartir servicios o solicitar ejecutarlos. Esto es un punto importante ya que nutriría al sistema de incentivos para que cada agente (nodos y servicios) interaccionasen sin un objetivo común, logrando asi alcanzar, o almenos acercarse, al concepto de autómata celular.

Además de estos dos puntos anteriores, tambien sería conveniente mejorar la especificación de la interfaz o la arquitectura de un servicio, asi como permitir que varios servicios compartan memoria sin que uno tenga que depender de otro (especificar una base de datos distribuida), añadir a nivel de nodo un protocolo que permita distribuir archivos con mayor eficiencia y flexibilidad que hacerlo por RPC entre pares de manera aislada, como por ejemplo IPFS, con el que se podrían compartir especificaciones de servicios, contenedores o modelos, o permitir la conexión entre nodos sin configuración previa.

Por otro lado, con el fin de mejorar la usabilidad y transparencia tanto del nodo como del servicio de clasificación se podría añadir una API para extraer estadísticas tanto del clasificador como del nodo. Asi como un cliente web para interactuar, por un lado, con el clasificador, con el que ver las métricas relacionadas con el estado del servicio o estadísticas de los servicios entrenados, y por otro lado con el nodo, con el que ver los servicos que instancia y sus dependencias de manera gráfica o sus pares en la red.

Tambien quedaría como tarea pendiente varificar la integridad y calidad de los datos que puedan ser proporcionados por parte de un cliente al clasificador. Este requeriría posiblemente provar estos datos durante varias iteraciones, lo que supondría un coste computacional elevado.

Otro punto más que no se ha desarrollado es el de habilitar la modificación de variables de entorno en los servicios. Esto permitiría modelar un servicio y abriría las puertas a, por ejemplo, que durante el entrenamiento se comparasen varias configuraciones de cada servico, o incluso se fuera alcanzando las más idoneas mediante, por ejemplo, un Algoritmo Genetico.

References

[1] Deterministic and minimal docker images. `https://gregoryszorc.com/blog/2014/10/13/deterministic-and-minimal-docker-images/`.

[2] Documentation grpc. `https://grpc.io/docs/`.

[3] Envoy proxy. `https://www.envoyproxy.io/`.

[4] Evolución de los sistemas distribuidos. `https://prezi.com/aswklby3ic58/la-evolucion-de-los-sistemas-distribuidos/`.

[5] Flatbuffers documentation. `https://google.github.io/flatbuffers`.

[6] Gunicorn python. `https://gunicorn.org/#docs`.

[7] Ipfs documentation. `https://docs.ipfs.io/`.

[8] Isa-json format. `https://isa-specs.readthedocs.io/en/latest/isajson.html`.

[9] Open container initiative. `https://opencontainers.org/`.

[10] Open neural network exchange. https://onnx.ai/.

[11] Protocol buffers | google developers. https://developers.google.com/protocol-buffers.

[12] Python 3.9.3 documentation. https://docs.python.org/3/.

[13] Rest vs grpc. https://es.accentsconagua.com/articles/code/rest-vs-grpc-battle-of-the-apis.html.

[14] Scikit-learn.

[15] Sistemas con estado y sin estado. https://www.redhat.com/es/topics/cloud-native-apps/stateful-vs-stateless.

[16] threading - gestionar operaciones concurrentes dentro de un proceso. https://rico-schmidt.name/pymotw-3/threading/.

[17] Welcome to flask. https://flask.palletsprojects.com/en/1.1.x/.

[18] What is faas. https://www.redhat.com/es/topics/cloud-native-apps/what-is-faas.

[19] Bitcoin key generation. https://www.oreilly.com/library/view/mastering-bitcoin-2nd/9781491954379/ch04.html#:~:text=The%20bitcoin%20private%20key%20is,generated%20from%20the%20private%20key., Jun 2020.

[20] ¿qué es kubernetes? https://kubernetes.io/es/docs/concepts/overview/what-is-kubernetes/, Jun 2020.

[21] A beginner's guide to understanding and building docker images. https://jfrog.com/knowledge-base/a-beginners-guide-to-understanding-and-building-docker-images/, Mar 2021.

[22] Docker documentation. https://docs.docker.com/, Apr 2021.

[23] Documentacion ngnix. https://www.nginx.com/, Apr 2021.

[24] Ramanan Balakrishnan. A simplified guide to grpc in python. https://www.semantics3.com/blog/a-simplified-guide-to-grpc-in-python-6c4e25f0c506/, Jan 2019.

[25] Nigel Brown. Explaining docker image ids. https://windsock.io/explaining-docker-image-ids/, Apr 2018.

[26] Davi Frossard. Linear regression with numpy. https://www.cs.toronto.edu/~frossard/post/linear_regression/, May 2016.

[27] Weiwei Gong and Xu Zhou. A survey of sat solver. https://aip.scitation.org/doi/pdf/10.1063/1.4981999.

[28] Pedro Gutiérrez. Para que sirven las functiones hash. https://www.genbeta.com/desarrollo/que-son-y-para-que-sirven-los-hash-funciones-de-resumen-y-firmas-digitales.

[29] Mano lingam. Hashes are unique. https://medium.com/predict/hashes-are-unique-9af14fe3796a, Apr 2019.

[30] John Minnihan. Understanding the docker cache for faster builds. https://thenewstack.io/understanding-the-docker-cache-for-faster-builds/, Mar 2021.

[31] Real Python. Linear regression in python. https://realpython.com/linear-regression-in-python/, Mar 2021.